Final Major Project – 7CTA1126 – Music & Sound Project

Keshav Deb - 15029277

Table of Contents

Introduction

For this project, I have decided to create a controller and software specifically for mastering use; allowing the user to master a whole song using their favorite plugins within this one device & software.

The aim of the device & software is for people to not only master using this one unit, but emphasis on the use of our ears with no screens just pots, sliders, and headphones/monitors. This also allows the ability for blind people to master, as I will implement braille to the hardware.

Plan

1. Research what is used in mastering and the main aim which is needed to be achieved within mastering.
2. Research similar hardware/designs and plugins.
3. Coming up with a design which is feasible to incorporate braille, pots, sliders, and conductive paint (instead of using physical buttons).
4. Research on Max/MSP capabilities to create this physical hardware and plugin.
5. Research Arduino connectivity for pots, sliders, and conductive paint.
6. Creating the physical device.
7. Creating a User Manual / Instructions.
8. Testing the device with a student, teacher and mastering engineer and receiving feedback (all test users will be mastering the same song).
9. Reflect on success/failure and conclude.

Research

Braille

Braille is a system of raised dots used for people who are blind or have low vision to enable them to read via touch, (What Is Braille? | American Foundation for the Blind, n.d.).

There are two main forms of braille, Uncontracted and Contracted, previously known as Grade 1 and Grade 2, respectively.

Uncontracted Braille is the basics, where every word is spelt out and most people learn this first before they learn the contracted version, (12 things you probably don't know about braille, n.d.). Contracted braille is like a shorthand version and is used by more experienced users and tends to be more commonly used to save space within books and other material etc. "It adds a series of specials signs to represent common words or groups of letters" as stated in (Contracted (grade 2) braille explained, n.d.).

Although largely similar there have been differences in braille within the English language across English speaking countries. Mainly between the US and UK versions. Work on Unified English Braille (UEB) started in 1991 by the Braille Authority of North America, (Simpson, 2013, p.xi). Overtime it grew international interest and after fully being developed Australia adopted it first in 2005 then followed by Canada, UK and finally USA by 2012 (Unified English Braille (UEB), n.d.).

I have chosen to use uncontracted (grade 1) braille (due it being widely used as the basic) and specifically Unified English Braille Code 1. I will be using online braille translators for my product and will cross examine the translation with multiple online translators to ensure the translation is correct.

List of translators that were used:

1. https://www.brailletranslator.org/
2. https://wecapable.com/braille-translator/english-to-braille-converter/
3. https://twoblindbrothers.com/pages/braille

List of Used Braille Words in product:

- Gain = ⠛⠁⠊⠝
- Equaliser = ⠑⠟⠥⠁⠇⠊⠎⠻
- Compressor = ⠉⠕⠍⠏⠗⠑⠎⠎⠕⠗
- Limiter = ⠇⠊⠍⠊⠞⠑⠗
- Threshold = ⠞⠓⠗⠑⠎⠓⠕⠇⠙
- Ratio = ⠗⠁⠞⠊⠕
- Attack = ⠁⠞⠞⠁⠉⠅
- Release = ⠗⠑⠇⠑⠁⠎⠑
- Q = ⠟
- Freq = ⠉⠗⠑⠟
- Ceiling = ⠉⠑⠊⠇⠊⠝⠛
- Low = ⠇⠕⠺
- L Mid = ⠇ ⠍⠊⠙
- H Mid = ⠓ ⠍⠊⠙
- High = ⠓⠊⠛⠓
- Drive = ⠙⠗⠊⠧⠑
- Dry / Wet = ⠙⠗⠽ ⠇ ⠺⠑⠞
- A / B = ⠁ ⠇ ⠃
- Volume = ⠧⠕⠇⠥⠍⠑
- Stop = ⠎⠞⠕⠏
- G = ⠛
- EQ1 = ⠑⠟⠼⠁
- C1 = ⠉⠼⠁
- SA = ⠎⠁
- EQ2 = ⠑⠟⠼⠃
- C2 = ⠉⠼⠃
- SI = ⠎⠊
- L1 = ⠇⠼⠁
- L2 = ⠇⠼⠃
- Play = ⠏⠇⠁⠽
- Pause = ⠏⠁⠥⠎⠑
- Forward = ⠋⠕⠗⠺⠁⠗⠙

- Back = ⠒ ⠄ ⠤⠒
- Stereo Imager = ⠒⠄⠒⠤⠒⠤⠒ ⠐⠄⠒⠄ ⠠⠤⠒⠤
- Saturation = ⠒⠄ ⠒⠂⠒⠤⠄ ⠒⠄⠐⠒⠒

Design

The main design concept was going to implement the use of conductive pain and many pots. Since previously working with conductive paint, I decided to go for a similar approach with my FMP by using wood again.

Figure 1.0 – My Undergrad Final Major Project Final Design

My first draft, drawing out a layout of where buttons/parameters will be placed.

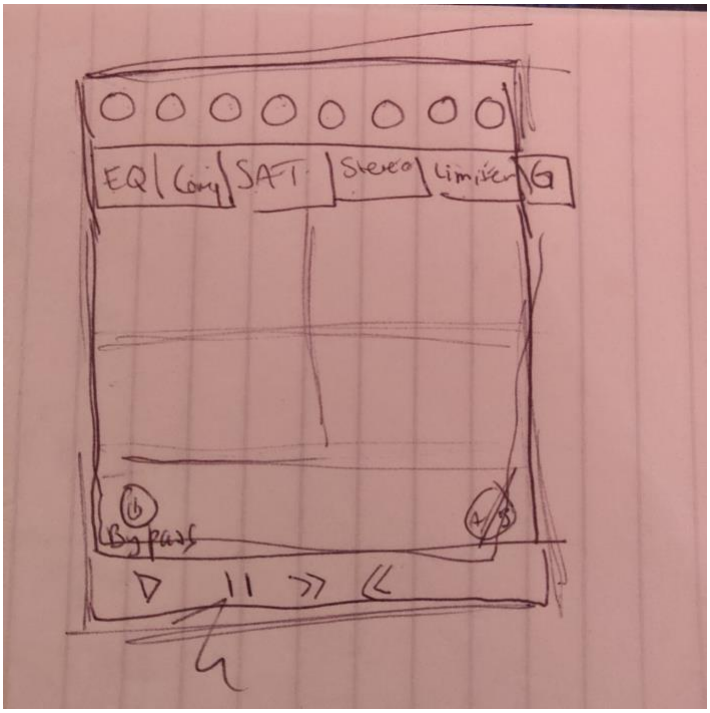

Figure 1.1 – First Draft

Here I was trying to figure out how many selection processers I would need and realised I missed out a gain parameter (as shown on top right), also how I would want the transport bar to look like and what it should consist of.

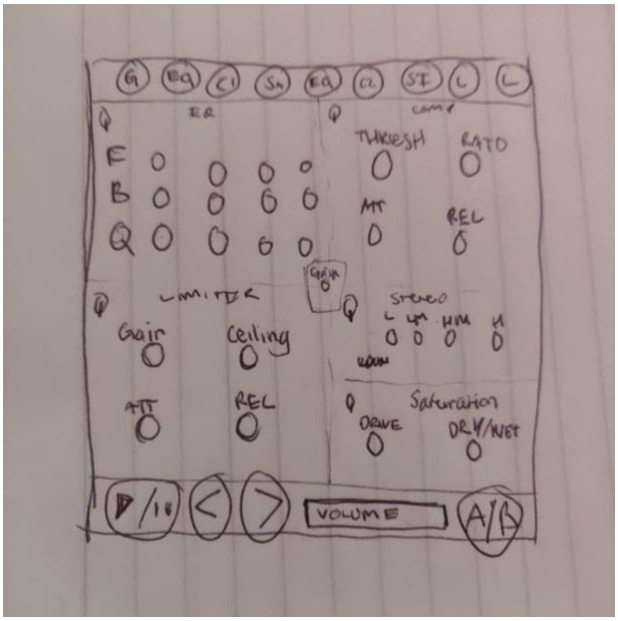

Figure 1.2 – Second Draft

In this second draft I figured out the final product and the placement of where things will be and how many parameters will be needed. In addition, just after drawing this I have decided when creating a digital version, I will keep all the on/off buttons on the left or right-hand side respective of which side the processing is on.

Figure 1.3 – Example of Ozone 8 Layout

Inspiration was taken from iZotope Ozone 8's layout with having the signal chain at the top and underneath that is the selected processing, as shown in Figure 1.3. In addition, also more precise inspiration came from Lurssens Mastering Console, with all processing parameters constantly out in the GUI, as showing in Figure 3.1.

Arduino & Components

For the build I will be using an Arduino Mega due to the number of I/Os needed.

Parameters:

1. 27 Potentiometers
2. 1 Slider Potentiometers
3. 19 Touch Sensor Buttons (Conductive Paint)
4. 2 Multiplexers

Due to the amount of I/Os, which are 28 analogues and 19 digitals. There needs to be two 16-channel multiplexers to spread the analogue inputs, due to the limited number of them in the Arduino Mega.

I have chosen to use 10k Potentiometers for several reasons. The lower the resistance, the more power it pulls out of the power supply and the higher the resistance the more the device is susceptible to noise. Due to the number of pots and buttons, I felt that 10k pots will be the reasonable balance.

I considered a 1k pot, but because of the Ohms law:

(Current (Amps) = Voltage/Resistance)

1k Pots:
5/1k = 5mA.

5mA x 28 Pots = 140mA

Which is already going over the power consumption (this is without considering the buttons).

Whereas,

10k Pots:
5/10k = 0.5mA

5mA x 28 Pots = 14mA

This is more than enough to also include power needed for the touch sensors (calculations was done using the four 5V pins that the Arduino Mega has).

I will be using solderless breadboards for this DIY project; multiple sets of wires and jumper cables to connect everything together.

Furthermore, after speaking to an Arduino/Electronics expert they said due to the amount of capacitive sensing there will be with the conductive paint it is better off to use PCB touch sensor buttons, which I also ended up ordering. This is due to the amount of conductive paint that will end up being used and the issue with them possibly crossing over when picked up via the Arduino. My previous issues from my final major project with the reliability and how inconsistent they could be was also a problem that I could re-face. Furthermore, there needs to be a good spread of paint for it to also work properly and I was thinking to use them like the size of a 20p coin.

Next, I created a wiring diagram for my proposed plan of the build, as shown in Figure 1.4.

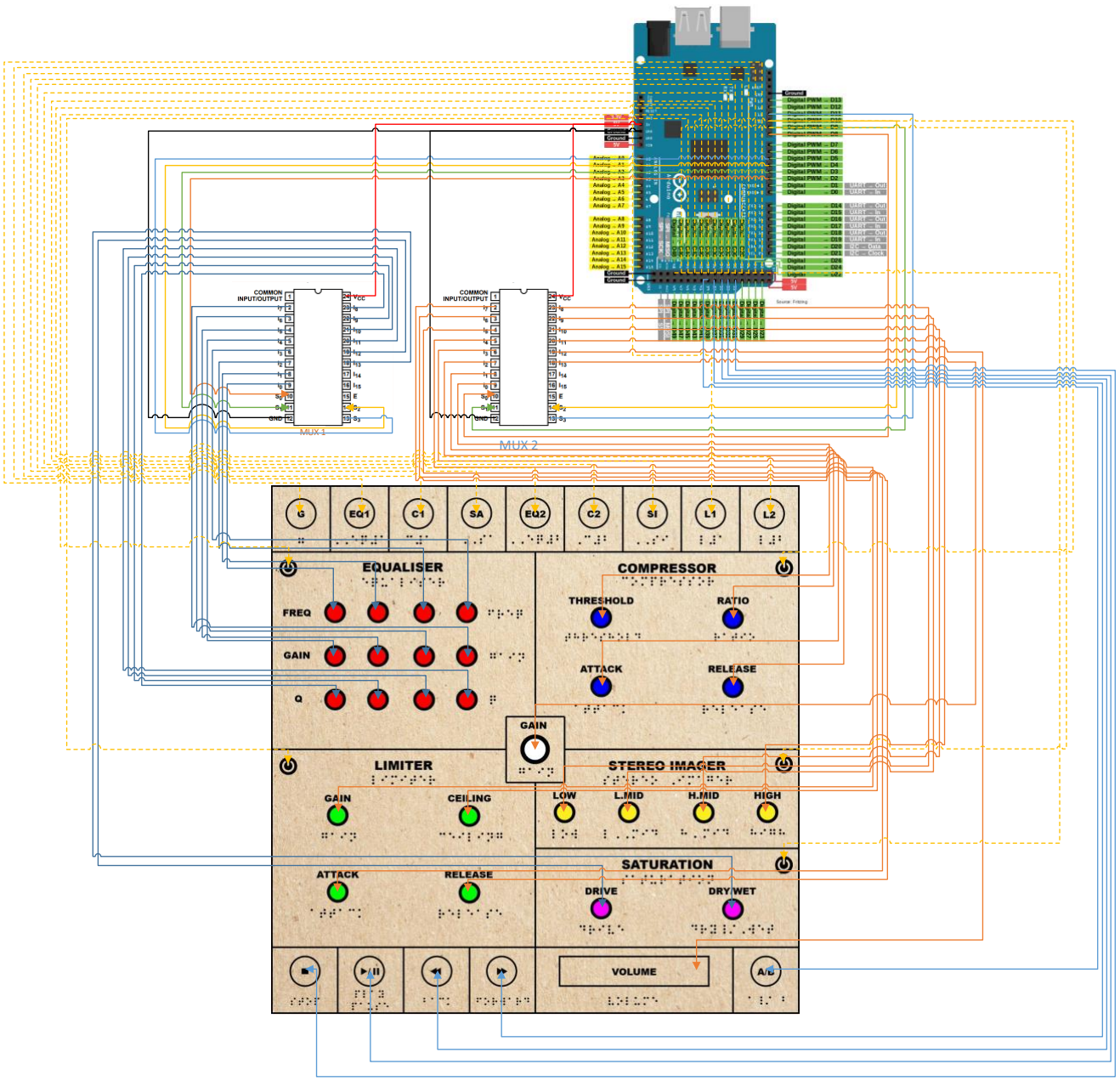

Figure 1.4 – Proposed Wiring Diagram

Max/MSP

For the Max/MSP research I used a combination of YouTube tutorials, Max/MSP forums and found the object help files very useful in terms of how to code certain things.

Most of the coding came from previous experience within using Max.

However, a few things which I came across through the forums made the process much smoother in creating certain code that I needed.

The main issue I needed to find out was how a Plugin works within Max and how much we can interact with it. Through YouTube I found out there was a whole section which you were able to drag and drop your plugins from the left-hand sidebar. You can choose from the vst plugins that you have – thus creating an object called vst~ (when dragged in).

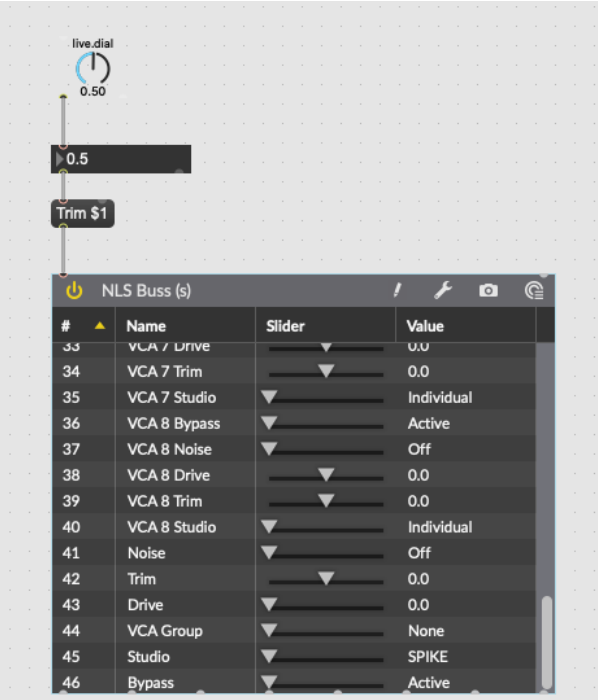

Figure 2.0 – vst~ testing with parameter change

Whilst testing this object I realised all parameters can only be changed with a float number from 0. To 1. and all decimals in between will translate to the plugin. In Figure 2.0, I am control the trim object using the live.dial and that 0.5 relates to 0dB in the plugin and if the dial was at 1 it would be +15dB which is the maximum on the NLS buss (chosen plugin in example).

Another important factor was being able to capture the information of an increasing value of a float number. Found in a forum was an example of an object I was unaware of called f object (float).

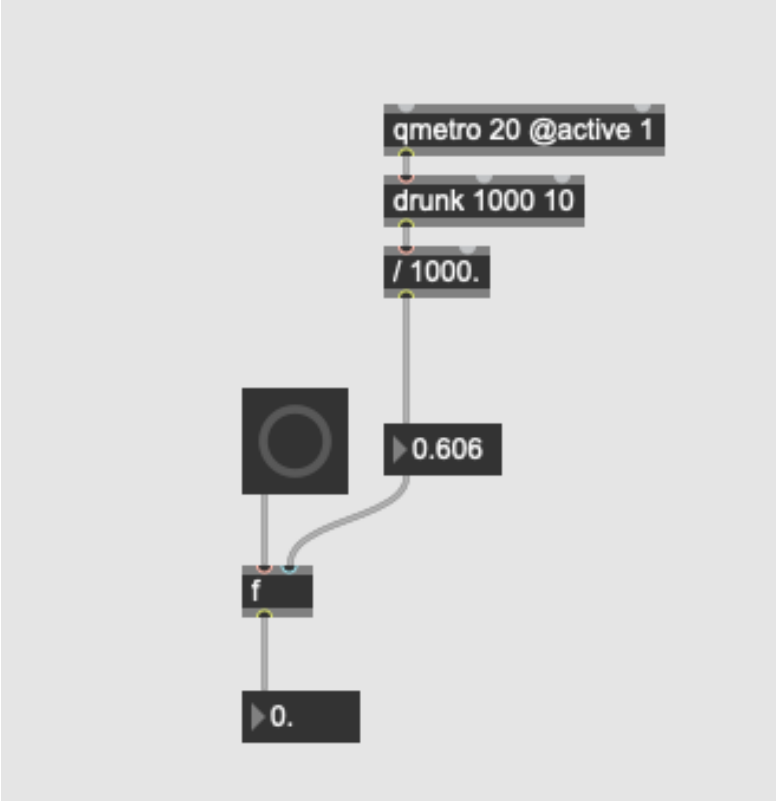

Figure 2.1 – f~, float object capturing moving value

This object allows me to use a bang to then send through the exact number the moment the button was pressed. Through this it let me obtain the exact milliseconds we are on to create my forward and backwards buttons.

The second issue I ran into was figuring out how to switch between the master and reference song, all whilst using the same transport controls and without the reference going through the chain. Found in the Max/MSP help forum was a patcher that describes a way to implement two "stereo" sources to switch between using object 'selector~ 2 1'.

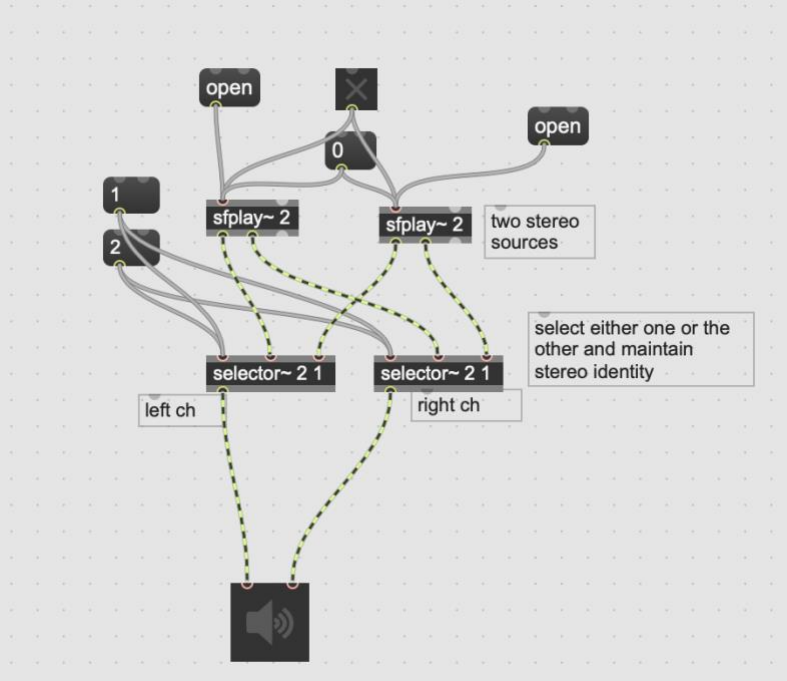

Figure 2.2 – two selector~ objects creating a stereo A & B switch

The 2 in the selector object is for having 2 inlets for the stereo signals – this is to send the left channel of a stereo file to one side and the right to another as shown in Figure 2.2. Therefore, the first inlet allows you to choose between 1 and 2 , because of this I created an 'if' statement later, as a toggle switch (Labelled as A/B) only works with 0s and 1s to transale 1 and 2 to 0 and 1.

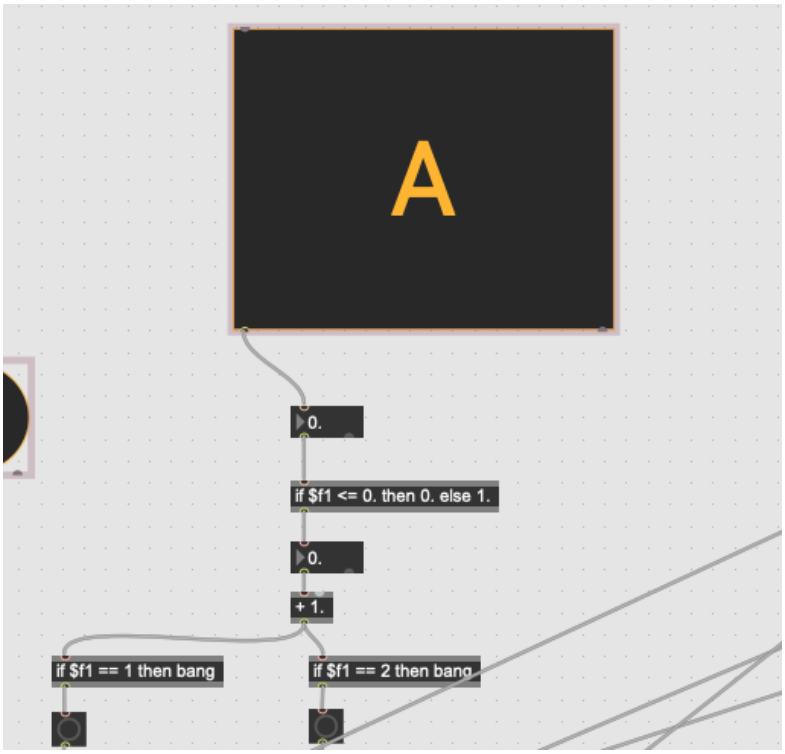

Figure 2.3 – if Statements

In Figure 2.3, we can see that I had to add an addition object "+ 1" to translate the 0 and 1, to work with 1 and 2, along with the 'If' statement. Furthermore, similar/variant of the if statement was used with another toggle (Pause/Play button).

Example Hardware/Plugin Products

SSL UF8

The first hardware product I found is an advanced DAW (capable of controlling: Pro Tools, Logic, Ableton Live, Cubase and Studio One) controller by SSL – model UF8; this device is mainly aimed towards mixing via this one controller without the use of your mouse/keyboard (as much as possible).

They have their own software called SSL 360; through this they can set user assignable controls. Eight 'Soft keys' which are buttons located at the top of each channel on the UF8, "each can have up to five assignments, which are switched in banks…each layer thus can have up to 43 assignable controls, not counting the footswitches" as quoted by (Inglis, S. 2021). The importance of these keys and what makes the UF8 special is that they can create macro like functions in DAWs that do not support macros.

Since there is no specific transport bar in the UF8 design, the soft keys are usually set for this, for you to use play, pause, and stop. In addition, to scrub through the playhead you must use the large rotary knob on the bottom right-hand corner, as shown in Figure 3.0.

Figure 3.0 – SSL UF8 Design

In terms of the transport bar, I would prefer a dedicated section for this in my hardware/software, in comparison to having it spread around like the UF8, where it must be assigned and shared with other buttons in banks.

The idea of using buttons which control many things in one press (a macro) will come in use within Max/MSP, as one button will have to trigger many setting changes (also within the code) in terms of selection of plugins and even make audio play out loud – to help the blind clearly hear which selection has been made, when pressed.

Lastly, I had two paths to choose. Either Max/MSP to control a chosen DAW (Logic Pro X) or to do everything within Max/MSP. In comparison to this DAW controller, I have chosen to do everything within Max/MSP, due to the connection from DAW to Max/MSP being temperamental with previous experience. Furthermore, making the software by itself, match my Mastering hardware controller, having less setup/programs will make it easier to use and less strain on CPU.

iZotope Ozone

This is a single plugin which contains all necessary tools to Master a whole song. Within the plugin it has 'processers' at the top, which is essentially your signal chain, thus can be changed and re-arranged. Beneath this you have full control of a bigger window of each plugin once selected at the top.

I translated this similar idea/arrangement into my build/software. With selective processers at the top then beneath you have your parameters of each process. Only differences are that I will have no metering due to contextualisation of the blind and enabling the user to solely use and train their ears (without the influence of visual readings). Lastly, other similarities also include being able to use iZotope Ozone without a DAW, as a stand-alone and being able to A/B with a reference track on the go.

Lurssen Mastering Console

A mastering plugin which can be used as a standalone or within a DAW, created by IK Multimedia. Lurssen Mastering is based in California, ran by Gavin Lurssen and Reuben Cohen (grammy-award winning mastering engineers). The idea of the 'Lurssen Mastering Console' plugin is to "replicate Lurssen's, whole is greater than the sum of its parts philosophy, which focuses on the order, connections and interactions between each individual processing component in the chain", as quoted in the press release (IK Multimedia releases Lurssen Mastering Console, 2016).

This was an important factor in my process of deciding my final signal flow and how one processor can affect how another works; when it comes to how I master myself within multiple genres. This is a similarity that I liked within how Lurssen works, because they created something that follows how they master. What makes them a step further in terms of being unique, is that the software "recreates the physical and audio-interactions between them in the chain" (ibid, 2016). Whereas most mastering consoles or processors just focus on the modelling of the individual processors. This is something I cannot recreate in terms of the modelling of the interactions and variables in-between because I will be using already made plugins; the focus of the order is more important in this project.

In terms of design, which is also like iZotope Ozone, but even simpler which is the approach I am going for. In comparison to Ozone where you select a single processor then you get to control the parameters individually; Lurssen has all processing controls out in the GUI. Another similarity includes inputting the song at the top left which I will implement, as a full stand-alone product, all shown in Figure 3.1.

Figure 3.1 – Lurssen Mastering Console GUI

Mastering Research

Mastering is simply the final step in the creative process (Owsinski, B. 2017, p.18), to achieve a balanced tone, maximizing the overall level to compete within general industry levels (dependent on genre), ensuring all codes and data has been inputted, the time to make any final fixes and understanding how the master will sound distributed across different platforms (taking account how of how songs will be encoded in different formats).

The process has changed overtime from 1948, the magnetic tape recorder needed to be transferred to a vinyl master for delivery – named transfer engineers. But there was an issue in which in the transfer process, they had to watch out for loud transient peaks, these peaks could cause the stylus to pop out and destroy the disc, this was expensive at the time (ibid., p19-20). To avoid this happening, the "dynamic processing tools such as compressors and limiters were introduced" (What is Mastering and Why is it Important? 2014). Furthermore, due to the introduction of the standardized RIAA Curve, the curve could enhance high-frequency transient peaks and could boost low-frequency energy, causing the stylus to pop out of the groove. Therefore, the application of EQ would fix this and allowed records to be cut with a narrower tighter groove, meaning longer playing time (ibid).

The 3 elements of EQ, compression and limiting are considered the main fundamental components of the mastering art form. These tools were used practically, but further down the line was utilized creatively to further enhance consumer listening (ibid). Currently mixing in the "box", you will see mastering engineers sometimes use saturation, stereo imaging and other extra processors to new creative process of mastering; this is the reason behind the chosen plugins in the list below.

EQ

I have decided to make use of a Linear-Phase EQ, this is because it reduces colouration through having a high resolution which "increases the smoothness" (Owsinski, B. 2017, p.51) and is much more transparent. These type of EQs are more commonly used within mastering, as we are trying to achieve a tonal balance and not change the overall mix. A Linear-Phase EQ also makes sure there is no delay added, in comparison to a regular EQ, when used it "adds latency to parts of the sound – this creates phase smearing. Certain harmonics

get delayed "smearing across" across the sound. Linear Phase EQs make sure the entire sound gets delayed simultaneously" as stated by (Swisher, n.d.).

Places where you would want to use this would be where there is a multi-mic recording (drums) or in this example mastering a one track where you don't always want to change phase relationship between different sounds (although sometimes this can sound good in different situations). In addition, this type of EQ is not great in terms of shaping the tone (therefore, we will have the analogue one to add warmth/character after).

Lastly, Linear-Phase EQ's tend be CPU intensive, therefore it is used mainly in Mastering compared to mixing (Keely, 2017). This can end up causing a problem called "Pre-Ringing" – "an artifact from the audio being processed backwards. It sounds like a reverse reverberated version of the sound spliced at the beginning of the transient", as quoted by (Swisher, n.d.).

A very important factor of Equalization during mastering is the acoustically treated room and accurate monitoring. Due to this hardware piece being made of wood it will be light and portable like any other small to medium sized controller. Thus, being able to take this to different studios and even master/reference on the go – the importance of listening to your masters through not only the accurate monitors but also through speakers which a normal listener will hear through for example: car, airpods, earphones, portable speakers and many more.

When it comes to using EQ in any frequency, less is always more. Meaning usually, you would go up to a maximum of 1-3dB of boosting or cutting. Ideally professional engineers tend to even cut or boss rarely any more than 1.5dB (10 Tips of effective EQ during Mastering | Waves. 2017). The reasoning behind this is due to similar reasoning behind keeping things transparent, making a tonal change rather than changing the mix and adding colour – if you find yourself making big changes using EQ, it is most likely that It needs to be sent back to the mixing engineer to make changes.

So due to this the design of hardware/plugins of EQ's that are specific for mastering usually have stepped pots – increasing/decreasing in increments of 0.5dB or 1dB (Owsinski, B. 2017, p.50). An example directly from Owsinski is the Manley Passive Stereo Equalizer, as shown in Figure 4.0.

Figure 4.0 – Manley Passive Stereo EQ Plugin

Lastly, as mastering has become a creative process people tend to add EQ not only for a fix of tonal balance but also to enhance the characteristics of the track. In conclusion, the first EQ (Parametric) that is commonly used is more corrective and second (Analogue) one in chain is used for enhancement (How To EQ During Mastering, 2017).

Compression

A compressor is a tool which allows control of dynamic level/range. "Specifically reduces the difference between the loudest and softest parts of your mix", as stated by (Raman, 2019). Reasons as to why we may do this is to make a track/s glue together, more coherent, punchier and allows to bring the softer details to push through (Katz, 2002, p.111).

In terms of compression in mastering, the primary goal is to raise the relative level – which is "How loud we perceive the volume rather than the absolute level that's on the meter." (Owsinski, 2017, p.45).

Mastering engineers use "compression as a tool to change the inner dynamics of music." Quoted by (Katz, 2002, p.121). Therefore, in mastering we tend to use very low ratios (parameter which changes the output of input into a compressor) around 1.5:1 to 3:1, this is due to not wanting to squash the mix further, as it may already be compressed a lot; setting the threshold to have gentle compression to try keep the sound of original mix the same. In comparison to pushing the compressor where you can hear it work which will sound unnatural and squashed – this can be a good thing during mixing, as it can help to achieve a desirable tone/characteristic (Owsinski, 2017, p.46).

The main parameters when it comes to mastering compressors are the attack and release times, as very little can go a long way and make a huge difference. As we are going for a 'transparent' compression, setting the attack time just slow enough so that it allows the transients to come through but will still react to slower things like vocals and bass. Adjusting the release time along to the track to keep it punchy (ibid, p.67), slow enough to where it does not make the gain changes more audible – "Compressors that work best on the full program material…have very smooth release curves and slow-release times to minimize this pumping effect" (ibid, p.70).

Then after this is the importance of ensuring you increase the make-up gain slightly, to compensate for the reduction in level (although it will be very little, as we are not compressing hard). Furthermore, a slow knee setting is always recommended over a fast one, as this is way the transition of uncompressed and compressed happens smoother (slow knee), the transition is more pleasant to the ears and transparent (Kagan, 2020).

In conclusion, overall, most mixes come in very compressed and squashed (dependent on material). So, listening to the track first to see if it needs anymore compression is important, but also testing it to see if after setting the parameters to your liking, if the sound turns out to be something you are not after, then you may not need one or try a different compressor (ibid.). I will include the same parameters in my build excluding the knee control (pre-set will always be soft).

Saturation

Saturation is not commonly used during mastering. But it can be a handy tool, when needing to create a fuller and more present (perceived loudness) track. It can also come in play to be used to 'glue' a mix together just like a compressor would but coating it very gently over the whole track (What is Saturation for Mixing and Mastering? – Sage Audio, n.d.). This effect is applied across everything in the track, this creates cohesion. In conclusion, the parameters I will have for this will be "Drive" and "Wet/Dry Mix", to have full control over "coating".

Stereo Imaging

In terms of stereo enhancement during mastering, it is mostly used to fix stereo issues and not a common thing in a mastering chain but comes in handy. As, we are at the final stage of a track, we need to ensure that the mix has mono

compatibility and that sounds are not lost or too wide/too narrow. It is said that the mastering engineer should add what is necessary, that the mix may have lacked (How to Use Stereo Imaging During Mastering, n.d.).

Limiting

A limiter is like a compressor, but it has higher starting ratios, usually from 10:1 all the way to ∞ : 1; taming the loudest/fastest peaks through a very fast attack time (Messitte, 2018). It also raises the overall level and does this transparently (when used correctly). In terms of transparent – "allowing to lift the average level and prevent clipping, without hearing any adverse effects – any distortion, pumping or loss of impact" stated by (Audio Issues n.d.). This is achieved by setting a ceiling (what we "call brick-wall" limiting), this will stop any signal from exceeding the 'limit', this is usually set anywhere from -0.1 to -1.0db. Then the second parameter there is commonly on a mastering limiter is a "gain" feature which will result in the boosting of your overall level. A popular mastering limiter plug-in example is the Waves L2 (Owsinski, 2017, p.48).

Figure 5.0 – Plugin by Waves L2 (Limiter)

As shown in Figure 5.0, sometimes the 'gain' control will be replaced with a 'threshold' control. This works in a similar way as quoted by (Messettie, 2018) –

"Some limiters have threshold sliders dictating the level at which gain reduction begins; these sliders also affect the corresponding output gain, and as such, the signal seems louder the more you pull these sliders down."

Other common controls include 'release' and occasionally 'attack'. I will also be adding these, as they can have a big impact on the sound. Fast release settings can cause pumping effect just like a compressor, so to achieve transparency using a moderate time will be the best option when limiting. Using a fast attack to reduce the transients enough where it does not become lifeless (Soundways.com. n.d.).

In conclusion, these common parameters stated above will be used and the end goal for the limiter in terms of mastering should be invisibly taking off transients and increasing the overall level without any audible effects/artifacts.

Metering

I have decided not to include any form of metering within the hardware. This is due to taking the approach of completely relying on your ears for perceived loudness, "*it's been said that 95 percent of all mastering is in the ears, and not the tools*", as quoted by Glenn Meadows (Owsinski, B., 2017, p.17). The only form of following standard mastering procedures will be a button to A / B to compare with a genre related reference track, as guidance.

Signal Chain

A mastering signal chain is very important in the final sound of the master and how each plug-in process interacts with each other in a linear order. There is no fixed signal chain but from researching different signal chains the basic things needed as stated before will be EQ > Compressor > Limiter or Compressor > EQ > Limiter. So, this will be the basis of how I create my 'Universal' signal chain; it stems from the analog mastering era (Owsinski, 2017, p.75).

The EQ first in the chain is still popular in terms of 'cleaning/fixing' signal as stated in the EQ section. When this is first it can also have a negative impact on the signal when a compressor comes after, as a boosted frequency will trigger the compressor to have "unexpected results", (ibid.) For this reason it will be sometimes good to change the signal around (i.e., compressor first).

Owsinski states a general rule of thumb for compressor/EQ order is:

- Large amounts of EQ, then place compressor before.
- Large amounts of Compression, then place EQ before.

(ibid.).

However, a plus side to having the EQ first is after the 'corrective EQ' is done as stated before, this will pass through a much more balanced signal and avoid 'pumping' (Zambrano, 2019).

A limiter always comes last in a signal chain, as it stops anything clipping over the output ceiling.

An advanced signal chain (if needed) can vary, but from various sources they all tend to have similar trends in terms of order. Examples of this can be seen below:

1. Gain
2. EQ
3. *Stereo processing*
4. Compression
5. *EQ*
6. *Gain*
7. *Soft clip*
8. Limiting

Figure 6.0 – Signal Chain Example 1

Figure 6.1 – Signal Chain Example 2

Figure 6.2 – Signal Chain Example 3

From these figures above we can see there is a common trend of having EQ > Compressor then another EQ > Compressor, with the extras that you wouldn't have in a basic signal chain in-between (stereo processors, saturation and more).

In conclusion, the important factor is to understand what each process is contributing to the master and how/why they are affecting one another. There is never a fixed signal chain, and a lot of variables will determine the master (Best Mastering Chain – Sage Audio, n.d.). Therefore, I have chosen a chain that works on most songs and allows multiple different combinations for example

Compressor > EQ > Compressor > Limiter. Furthermore, all have been previously used in my own work, I have confidence that they will do the job that they need to, all processors do not need to be engaged, hence why there is pairs of the basic processing needed to master which is EQ, Compression and Limiting.

The signal path I have gone for will work on must songs:

1. Gain
2. EQ (1)
3. Compressor (1)
4. Saturation
5. EQ (2)
6. Compressor (2)
7. Stereo Imaging
8. Limiter (1)
9. Limiter (2)

This allows different combinations, as we spoke about before with EQ > Compressor or Compressor > EQ (and many more possible combinations).

Serial Processing

Serial Processing is something which is used often in mixing and mastering. It is a technique which involves not using a processor too hard with one plugin or unit but instead spreading the intended use between two or more of the same processors. By doing this it creates transparent results (Pack, 2020).

For this reason, I have added two of each basic mastering processors (EQ, Compressor and Limiter).

Serial Compression / Limiting is the main ones used in terms of mastering, as we are trying to achieve loudness to compete with industry levels. It can be used for large amounts of transparent processing. Furthermore, applying different types of compressors/limiters, as I intend to have one for tone and one to glue. With this you can shape transients and fatten the sound (ibid.).

Other common processing

Other processes include a De-esser, Reverb and Tape Emulator, they are usually used for certain specific issues in certain situations.

- De-esser – will fix/tame sibilance, cymbals, and other high frequency content instruments/issues.
- Reverb – can open the stereo field/depth further when used subtly in certain circumstances.
- Tape Emulator – this would be used like saturation.

(Zambrano, 2019)

In conclusion, I am choosing not to add these processes to my build, as they are used in special scenarios mainly. Whereas, saturation and stereo processing is more common

Hardware Build: Max/Arduino

For my hardware build I have decided to use Wood as the casing, which includes 19 touch sensor buttons and 27 Potentiometers and 1 Slider Potentiometer.

Design

Firstly, I created a detailed image with each section divided mathematically, whilst keeping in mind how this could be replicated into a hardware design. I took inspiration from my previous Undergraduate Final Major Project, using wood and paint in terms of design, shown in Figure 7.0.

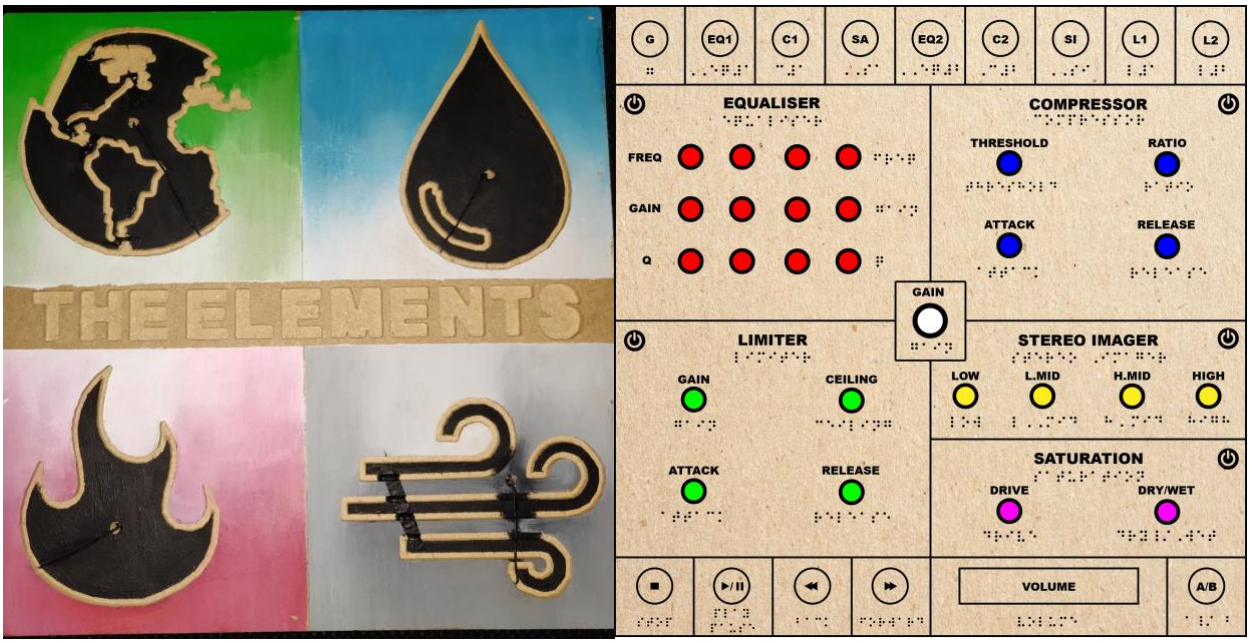

Figure 7.0 – Undergrad FMP Design and Illustrator Design of my idea

From my design created in Illustrator (Figure 7.0), we can see that everything is laid out in their own sections. The colored circles are the 27 potentiometers, the volume is the slider potentiometer and lastly, the transparent circles (including the on/off symbols) will be the touch button sensors. The on and off buttons is the only difference from my second draft, as I placed them in the top corners of their respective sides.

The bottom is the transport bar, overall volume/monitor control and in the bottom right there is an A/B for doing comparisons with mastered tracks.

The top is a selector (also in order of the signal-chain), so the idea is when a processor is selected, only the selected pots will be in use; the rest of the pots will be disabled through the coding (so even if they are moved when it is not selected it won't change the parameter, resulting in no sound change). In addition, when then selected after the change, there is still no actual audible change until moved again.

The middle has the 6 different types of processing which include EQ, Compressor, Limiter (which will all control a pair of its processing as shown above) and a Gain, Stereo Imager and Saturation - which all also have its own bypass button (excluding Gain) so you can hear the difference you have made.

Lastly, underneath all labelled parameters you can see its braille equivalent, ensuring there is enough space in the design to be able to fit this, for users with disability.

Max Design:

Designing the plugin was straightforward, as I wanted to use the illustration I created. I did this by using the fpic object, inputting a version of my illustration but without the braille and then place buttons and dials on top of the current illustration. I went for a theme of dark grey and light orange buttons; I placed a standard panel object underneath so I can have buttons for loading the master, reference, export functions, toggle hardware and timers. Lastly, for the dials I changed the way they look using the inspector window (also done for other buttons etc.), making it larger and changing the colours of certain dials so you can see them under every colour clearly. All of this goes into presentation mode in Max MSP, where you can remove all tool bars and access to changing any code, when handing in. However, I found out later I could instead create a standalone application for my plugin, so that was not needed. In Figure 8.0, the full final design can be seen.

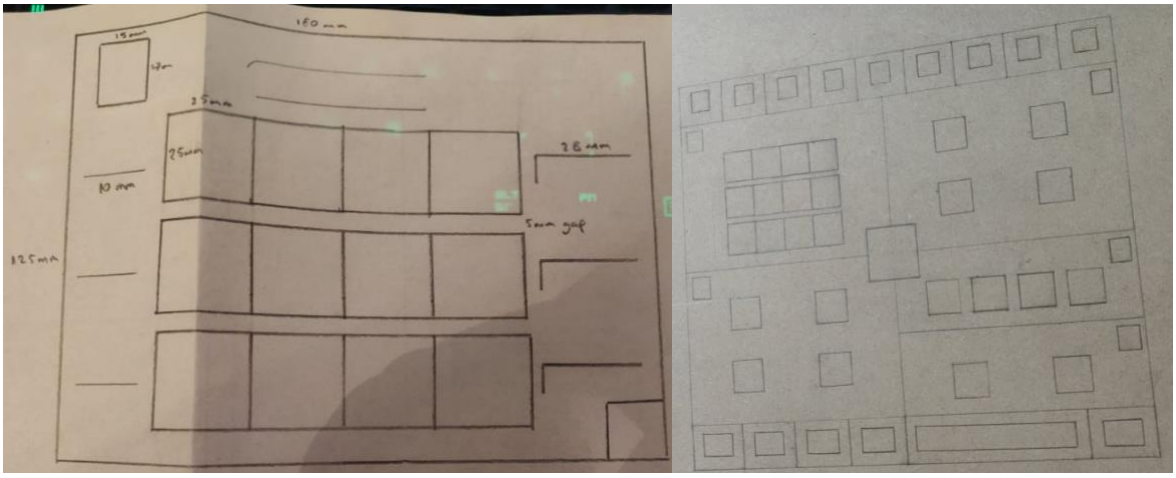

Figure 7.1 – Design & measurements of cut out

To translate the design from the illustration to the wood I had to size everything in ratio with the dimensions of the potentiometers, touch sensor buttons and braille text. This was then drawn onto the wood and then all hardware was placed. So left of Figure 7.1 will be the size of the Equaliser section and this will be the same size for each other section, I was able to build the selection and transport bar

after drawing the main middle sections. Also kept in mind was the spacing for braille and text. In terms of the braille, I left 5mm of spacing height-wise, as I found a custom braille/label printer that can print this, providing them a document of maximum length for each word.

The size of the touch sensor said it is 15 x 11 x 1mm. However, in the description of product says to please leave 1-3mm differs due to manual measurements (in Figure 7.2 we can see it ended up fitting perfectly).

The size of the buttons on top (signal chain) were also measured against the length of two of the photos above which will be 180 x 2 = 360mm. Therefore, each section will be 40mm in length due to dividing this by 9.

Lastly, the length of the slider had plenty of space either way due to the scaling of all the above.

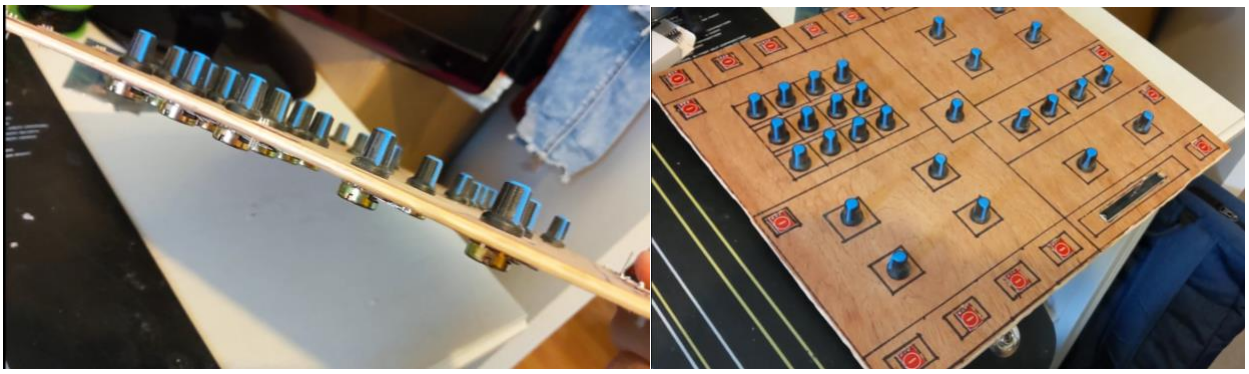

Figure 7.2 – Thickness of board & front

At first, 9mm thickness MDF was purchased but then I had to get 5mm hardwood plywood due to the clearance between the bottom of the pot and space to place the cover of the pot and allowing it to turn.

After testing on a 164mm x 52mm solderless breadboards; once wiring realized it will not fit within the back side of the board due to all the connections of pots, slider, buttons, and Arduino Mega sticking out. Therefore, I needed to purchase two 84mm x 55mm solderless breadboards; once everything was re-wired using the two breadboards, I then had calculations for the sizing of casing, also the number of wires that were sticking out at a certain height had to be considered, all of this can be seen in Figure 9.0.

Height: 323mm
Length: 360mm
Width: 150mm

These overall dimensions are not exact due to rough cutting of the wood. After, changing to Teensy microcontrollers, I built the sides of the casing with 5mm Bamboo Veneer Single Plyboard Sheet, these were very thin, so I had to hot glue the sides together and it seemed to have created strong sides. Furthermore, I drilled three holes to allow access to connect to the Teensy's and the power supply unit. However, only two of them will be used constantly, one to plug a micro-usb to the main teensy and one for power supply (third is just for access if any changes are needed to be made to the second Teensy).

Lastly, I had to create an opening for the back to get access to the main wiring, in case any changes needed to be made or parts needed to be replaced. I did this by adding two small hinges to connect to the back wood piece (a 3mm MDF sheet), which allows the sheet to open. I then added a cabinet roller catch to be able to keep the back closed, so wiring is not exposed. All this can be seen in Figure 7.3.

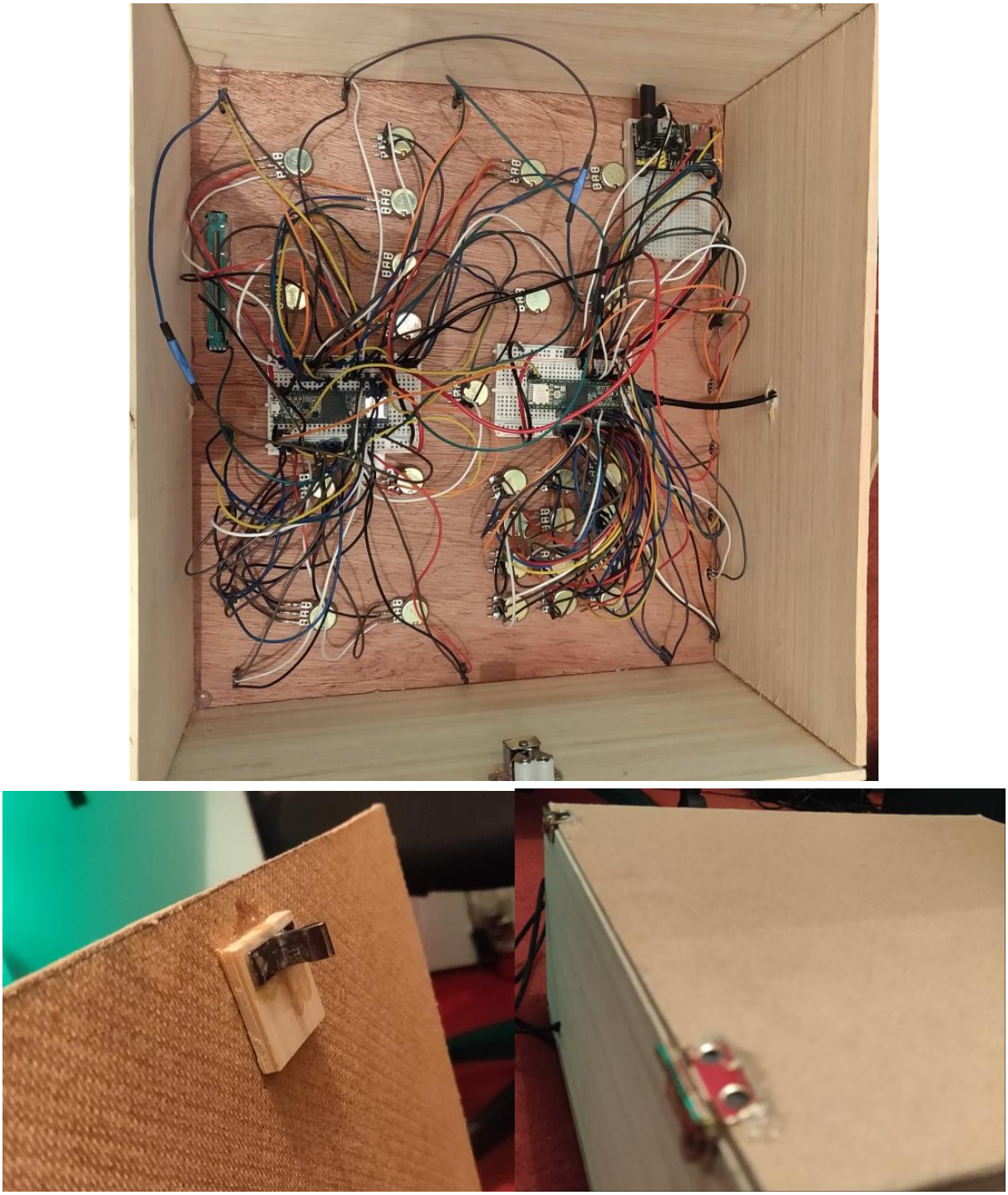

Figure 7.3 – Casing & wiring

The last thing added to the hardware design was all the braille labels and text, I ordered braille with a maximum of 5mm height and specific length for each word. I sent this to the custom label creator, and they came back with proofing which was then printed onto clear adhesive with black braille. The text was written onto wood by hand and final addition was a slider pot cap.

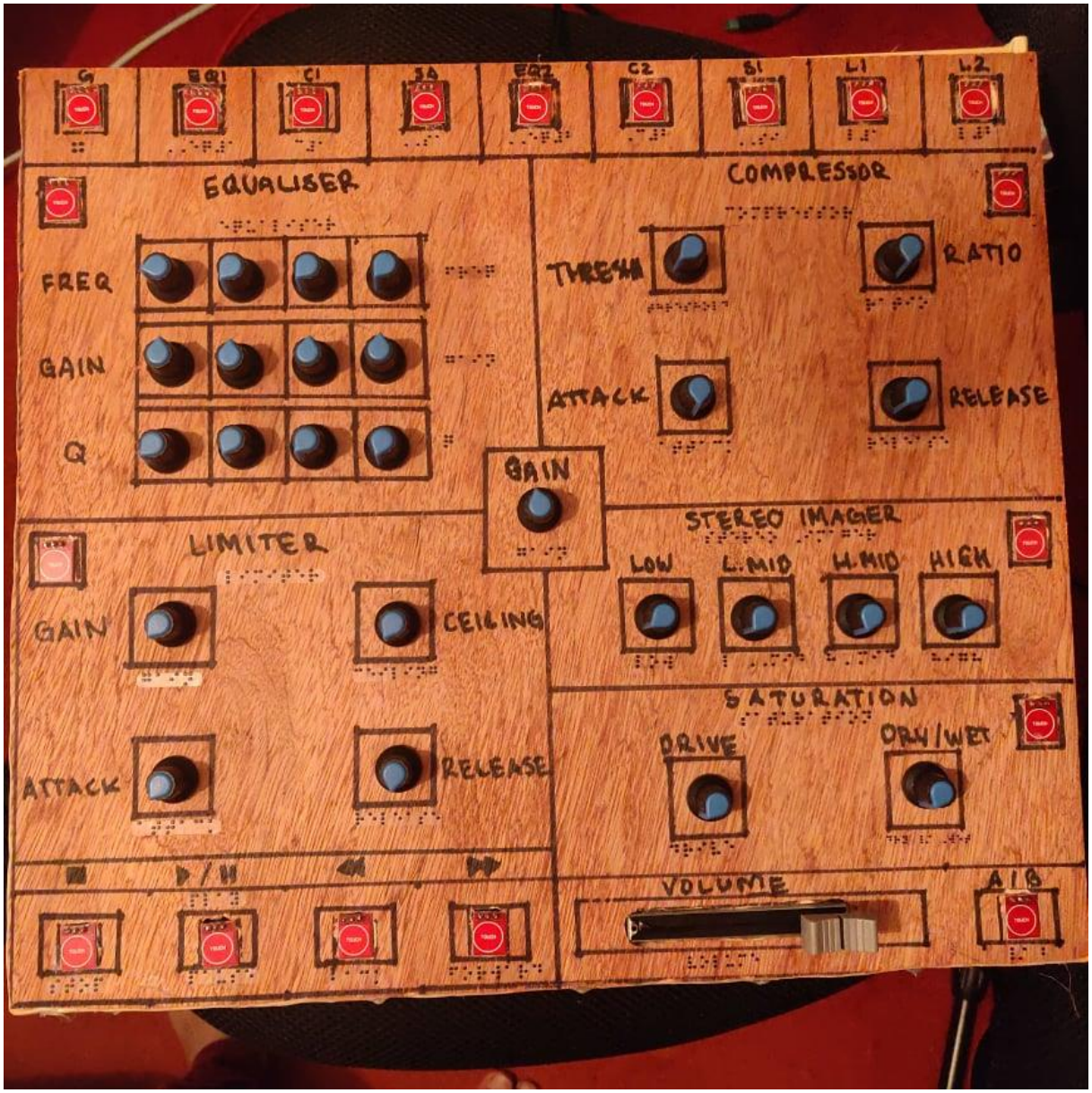

Figure 7.4 – Final front design

Max

Within max I created a graphic user interface to use in conjunction with the Hardware.

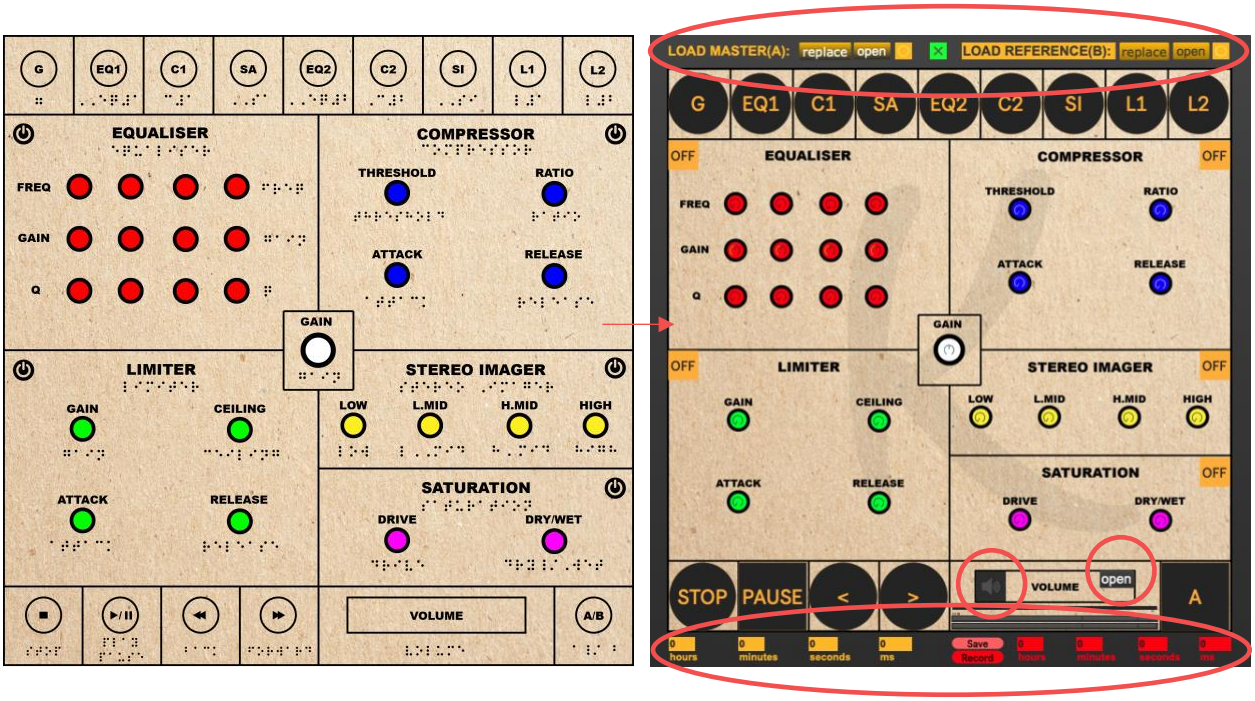

Figure 8.0 – Illustrator design created in Max/MSP as a full functioning software

All parameters work respectively with its hardware counterpart. For example, when a dial is changed on the hardware, the dial on Max/MSP should change simultaneously.

There are a few differences and the extras on the Plugin which include (circled reds on Figure 8.0):

- Load Master (A): Bang, Replace and Open – these are all to input the track you want to master into the system so it can be played using the transport bar. When loaded, it is run through the signal chain.
- Load Reference (B): Bang, Replace and Open – these are all to input the reference track into the system so it can be played using the transport bar. When loaded, it is **NOT** run through the signal chain.
- EZDAC - To turn sound, so there is playback.
- Playback Timer – Show current playback time in hours, minutes, seconds, and milliseconds.
- Toggle – Turns on metro, which is an on/off button for the hardware to send data through.
- Export/Bounce – Save, Record and Record timer, this function is for real-time exporting whilst also showing a timer to ensure that the recording is happening and the current time it is on.
- Open – to change input/output drivers and more audio statuses.

Transport Bar (BOTTOM):

This was created using sfplay~ 2 (stereo channel) for playback.

Buffer~ 2 and groove~ 2 with using myBuff as a common variable between these two to find the position of the song in milliseconds as a time measurement. This helps with seeking forward and back, as explained later.

Therefore, there are two of the same tracks being played simultaneously one for **playback** and one for **position**. However, one is muted and the one that goes through sfplay is being actively used.

Furthermore, I had to copy and paste the same code from the master track for the reference and during this I needed to ensure that there is another common variable for that, which is named myBuffTwo.

Open – To open chosen sound file to be loaded into sfplay~ 2

Stop – Button which sends a bang to 0 that is sent to the pause/play toggle when in "play" mode it will reset it back to pause (just like an old cassette/tape player). It also does the job of restarting the whole track from 0 milliseconds. Therefore, when also pressed, it sends a bang to 0 and then to sfplay~ (through pos $1) and 0 into groove~ myBuff to reset the song (also to the reference code).

Pause / Play – using two inlets of message "pause" and "resume from a toggle which outputs 0 or 1. An if statement was created to bang 0 or 1 accordingly to the required labelled pause / resume messages.

Seek Forward/Back – Since sfplay~ is for playback changing position of the track through this ranges from 0. – 1. in Max, as a float. However, obtaining a current position of playback can only be found through the buffer/groove objects. Buffer works with milliseconds, therefore using the object info~ myBuff, sending a bang through the 7th outlet outputs total time of the inputted file (which gives us variable $2 – which is the High input value). At the same time the position of track is being sent to the scale from groove~ myBuff into snapshot~ and *0. Multiplier (which gives us variable $1 – current sample position). This is then scaled from $1 to $2 to 0. – 1. This scaled moving value then is sent to the "f" object inlet 2, whilst either a

left or right button is pressed to send a bang to output the current sample the moment the bang was pressed. This information is then sent to either a 0.1 addition or subtraction dependent on forward or backward button being pressed (meaning seeking either goes up or down by 10% of the total milliseconds of file, which I have decided is enough to scrub through most song). Then this new value is sent to the pos $1 into the sfplay~ 2 which runs simultaneously. Lastly, the value is also then scaled back from 0. – 1., into the calculated equivalent sample – this ensures when pressing a seek button again it will all be in sync to find the next value.

Button/Bang – Both master and reference have a button. This is to send the total value of milliseconds from the inputted file into use for the rest of the transport bar to work.

Replace - To input chosen sound file to be loaded into "buffer~ myBuff 2". (2 to ensure stereo signal)

A/B – To use the AB technique – which usually entails using the reference track as a guidance of where you may want to be in terms of tone or loudness etc. This works by using the "selector~ 2 1" object to switch between the master which goes through the signal chain and the reference which goes straight into the selector then output (only allowing processing change in the master itself). Using an 'If' statement to change the toggles outputs of 0 and 1 to work with the selector, as shown in Figure 2.3.

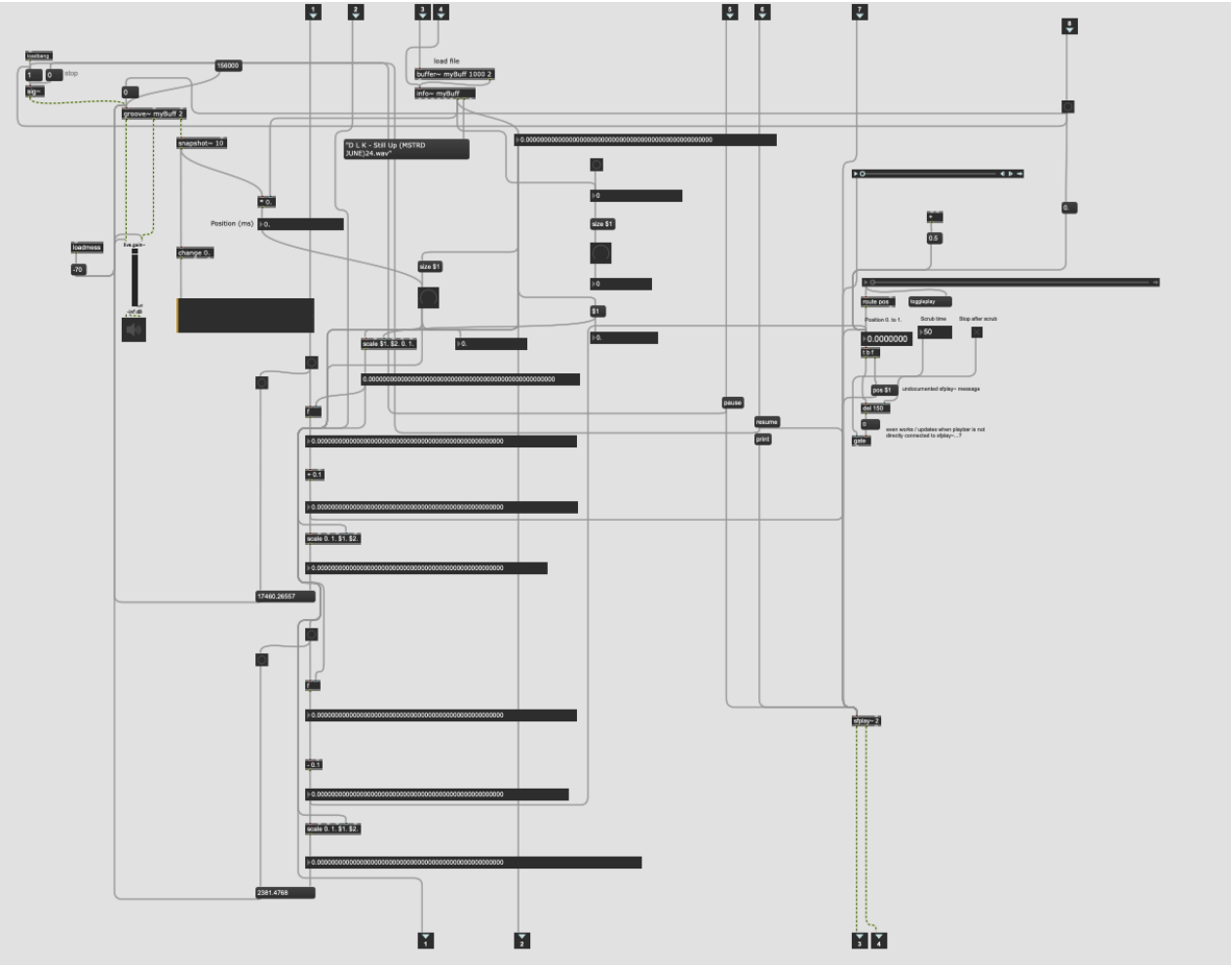

Figure 8.1 – Transport/Playback code for Master

Signal Chain/Selector Bar (TOP):

When a plugin is selected from the buttons, this disables all other dials excluding the chosen – this is all done via gates (object gswitch2). When an EQ, Compressor or Limiter is selected it works the same way. However, there is a master dial for the pairs and goes through further gates when process is selected.

The way to get around using two different processors (e.g., EQ1 and EQ2) with one set of dials is to use a gswitch2~ object this allows for you to press EQ1 and have the dials work for EQ1, so nothing in EQ2 changes unless you press EQ2.

Using this object after every dial, allows the user to technically disable any incoming movement from the dials that have not been selected. Therefore, resulting in no audible change. The good thing is, if it has moved and you select back to the one you moved whilst disabled it won't change the parameter to

where it has accidentally changed. This is because when the gate has changed the output goes to nothing.

The gates are changed using messages 0 and 1.

Figure 8.2 – Dials working with gates just before entering vst~

Processing Parameters:

The dials are sent into message boxes, which match the variable being changed in vst~ and the on/off buttons is sent into the message "bypass $1" to turn on and off the plugin. In addition, parameter values are all controlled in max from 0 to 1 and everything in-between as a decimal value (float), as we see in Figure 2.0.

Some dials have their range set. For example, the Gain dials for the EQ's to ensure it doesn't increase or decrease by more than 3dB for mastering purposes. In addition, the gain and few other dials are set at 0.5 (middle) as in the initial value, so when opened it starts in the middle just like the plugin does.

List of chosen *Plugins* with corresponding *Signal Chain*:

1. Gain = ***NLS Buss by Waves***
2. EQ (1) = ***FF Pro-Q3 (In Linear Phase Mode) by FabFilter***
3. Compressor (1) = ***SSLComp by Waves***
4. Saturation = ***FF Saturn by FabFilter***
5. EQ (2) = ***AR TG Mastering by Waves***
6. Compressor (2) = ***VComp by Waves***
7. Stereo Imaging = ***MStereoProcessor by MeldaProduction***
8. Limiter (1) = ***L2 by Waves***
9. Limiter (2) = ***FF Pro-L2 by FabFilter***

Playback Timer & Exporting Master

The playback timer was created by using the info generated from the 3rd outlet of groove~ into snapshot~ 1- into a "*0." multiplier object which then goes into a float – this shows position of playback in real time in (ms). This is then sent to a Number object (this is also done for the Reference Track). Which goes through a gswitch object to switch between Master and Reference playback positions using a bang to send the data through, as the A/B toggle is pressed. Lastly, this goes into the final step which is the "translate @in ms @out hh:mm:ss" object which then is sent to an "unpack 0 0 0 0" object, this outputs the respective numbers of hours, minutes, seconds and milliseconds.

To create the export function, I used object sfrecord~ 2 (for stereo) after the last limiter in the chain. This allows the user to record in the inputted signal in real time (i.e., no offline bouncing/exporting), so after you are happy with your master, you can take a ".aiff" file with you.

By sending message open to sfrecord~ 2, it will allow you to name a stereo file which you would like to record and save as.

Secondly, you would proceed to press the record $1 message which is also sent to sfrecord~ 2. The variable being the amount of length you want to record in milliseconds. As previously mentioned, before you start mastering, we obtain the total ms of the track via pressing the bang, this same information is also stored and sent to this record function. Once you press record, it will also trigger stop which means it will start recording from the top (as it will need to record live playback).

Furthermore, I have also added a timer to the recorder function to know start and end of recording. The output of sfrecord~ 2 goes into "snapshot~ 98" object, this reports at a rate of 98ms, which I found to be the closest rate that the number~ was outputting.

Lastly, on presentation mode I added separate buttons as 'Record' to press the record feature and 'Save' to the Open message – in different shades of red to show the difference between record timer and playback timer, as shown in Figure 8.0.

Button to Speech

I have implemented text to speech, respectively when each button is pressed the name of the button plays out the DAC all is connected to sfplay~ 2. This enables to ensure whichever button they have pressed will give an audible reference; this will be a great asset to the visually impaired users. This is all done by sending an open 'name_of_file.aif' into sfplay~ 2.

All recordings were done through, freetts.com. I found that the 'English (US)' language and 'en-US-Standard-C' voice, sounded the clearest to understand, when tested with all words.

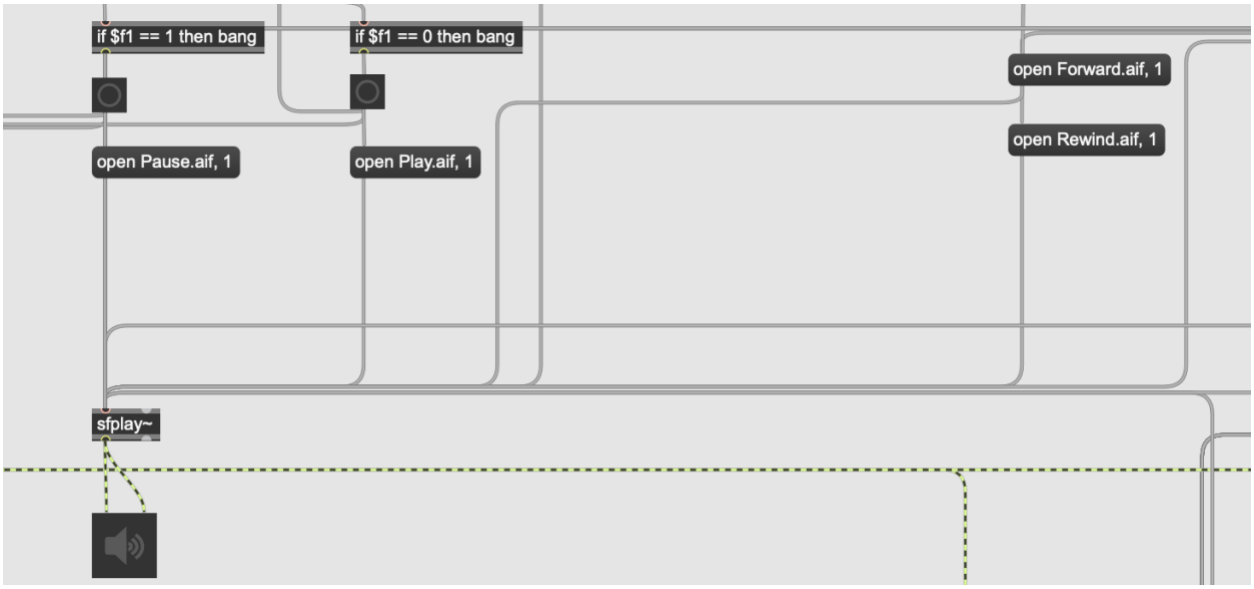

Figure 8.3 – An example of Pause, Play, Forward and Rewind going to sfplay~

Arduino to Max:

When connecting the microcontroller to max, using the serial object is the form of serial communication. Ensuring the rate is set align with the code in the microcontroller, therefore, the object would be serial 'x' 9600 (x, being the variable of which port, you have connected to within your computer – this can be found by sending a print message into the serial object), which means it can send 9600 bits per second.

For serial to work, it will need to receive a bang, so a metro object will constantly update the serial. I have chosen to set the metro object at 30ms, to have a constant sufficient stream of the serial.

When the raw information comes in (this can be seen in the Max console with a print raw object attached to the outlet of serial), it comes in the form of ASCII, so each digit will represent a single character and when there is a 13 and a 10, that will be the termination message. Therefore, using a select object combined with a zl group will output a string/stream of numbers. Within an object "sel 13 10", the third outlet states anything that is not 13 or 10 is our message, and every time you receive a bang that says 13, terminate the group. After the zl group, the ascii needs to be converted and this is done by using itoa object, for max to understand which will be a symbol. Then using fromsymbol object from the outlet of itoa, to make this into useable data that a number object will understand.

Lastly, using an unpack 47 object, this will separate the 47 streams of serial coming in – this matches the number of parameters I have. This can then be connected separately to its respective inputs.

All this can be seen in Figure 8.4.

For button objects it can connect straight to them, as its outlet is bang when a 1 is sent.

For toggle objects I created an if statement, where if $f1 == 1 then bang, this is to change states within the toggle from 0 to 1 only when the touch sensor is actively touched (when touched (LED ON) it outputs '1').

For the potentiometers and slider that ranges from 1023 to 0, a simple scale object will be used in align with the dials and volume. However, some of the inputs are not in the full range, especially all the EQ pots range on average around 990 to 0 (is very temperamental and changes often around that average of 990, usually dropping). In addition, the Ceiling pot ranges from 1015 to 190, which is a slight defect. **Note: this is very temperamental when tested a few days later, the ranges of those exact potentiometers decreased, and scale had to be changed again.** Anything from temperature, current and much more can effect an analog signal.

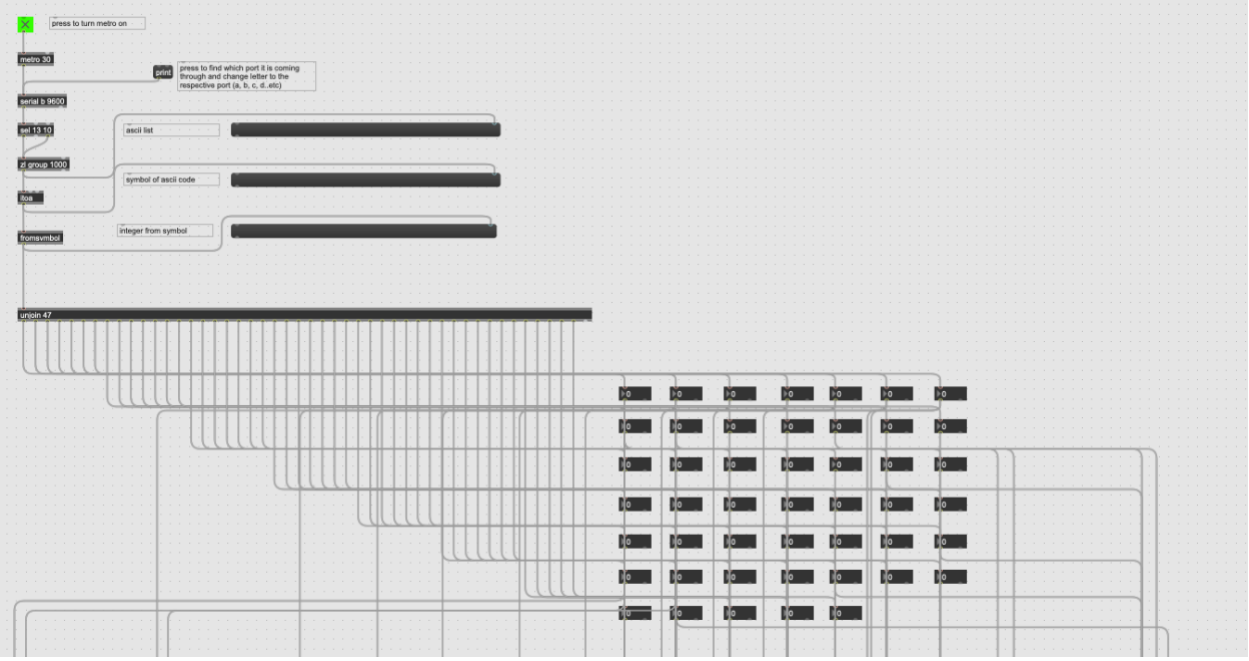

Figure 8.4 – Arduino to Max code

Standalone & Final Max Code

I created a standalone application using the Max File > Build Collective / Application, through this I added an extra code in the script named 'appicon' and then using 'Include File…' to select the .icns file, that is contained in the same folder (in this folder all audio assets for the plugin are all included).

Once opening the application, I noticed that it needed rescaling as 100% default zoom is too big, also little things needed to be changed like access to drivers, loading in presentation mode and on/off toggles need to open at "off" state.

All these issues were solved by using load objects (loadbang and loadmess).

We can see in Figure 8.5 that there are messages being sent to the 'thispatcher' object, 'presentation 1' makes it open in presentation mode and 'zoomfactor 0.3', makes the presentation mode open in zoom 30% which fits the GUI I have created perfectly (this then replicates when building application). I found for some reason that the zoomfactor message will not work unless there is a delay object before it.

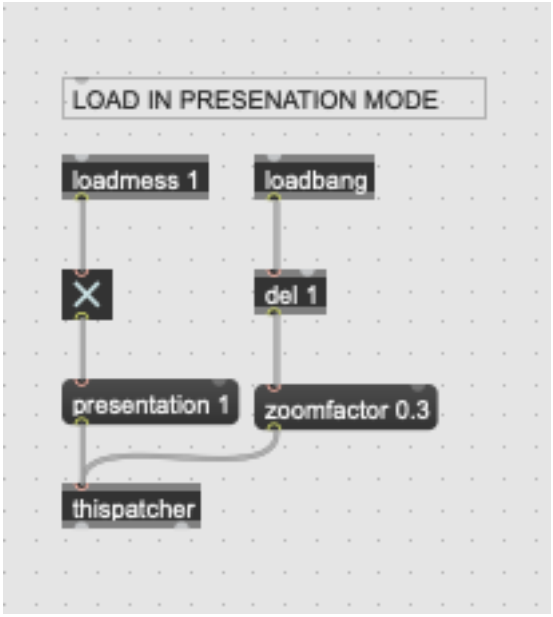

Figure 8.5 – Code for loading in presentation mode

Lastly, the same thing was done for the on/off buttons for every processor, a 'loadmess 1' object was used to set it as 'off', as a 0 message will keep the toggle as 'on'.

Arduino

When testing the wiring, I found there to be defective components (12 Potentiometers and 1 touch button sensor), the way this was tested was through using a multimeter to measure internal resistance, whilst turning the pots to see if the voltage changes – this would mean it is working. In addition, when connecting to Arduino and testing it in serial, I found that when there was a loss of current which also leads to a loss of voltage flowing through (due to the number of components there are), as the values should be 0 – 1024, but instead they decrease and become inconsistent. This is also due to the Arduino only being able to apply a certain amount of voltage, as the voltage regulator may possibly be limited in the Arduino Mega (even when attempting to plug-in a 9V power supply to the Arduino Mega).

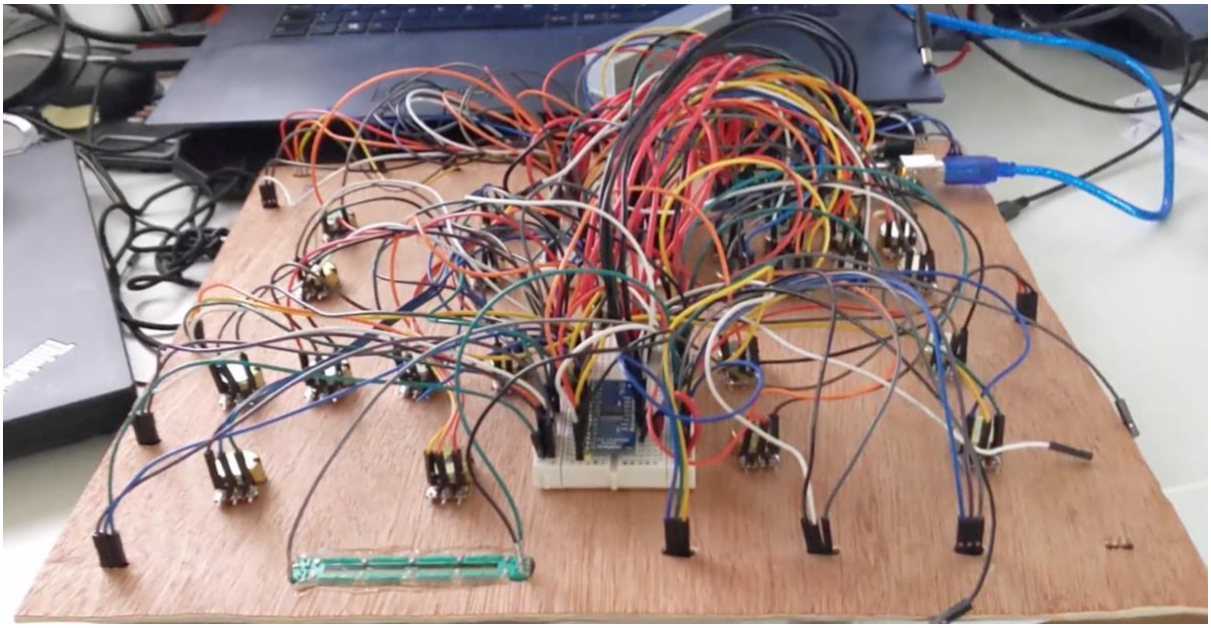

Figure 9.0 – Wires being tested and placement of breadboards

After speaking to an expert multiple times testing back and forth. There were multiple things I tried (in order):

1. Using the MUX breakout board instead of the integrated through-hole circuit (as show in in Figure 9.0 photo was taken after).
2. Buying a power supply for the Arduino
3. Trying a different microcontroller "Teensy"; not just 1 of them but implementing two to share the load. With the teensy a multiplexer won't be needed, due to having 42 breadboard friendly I/Os.

When testing the Teensy it seems to work much better, as it is more powerful.

The plan now was to use one Teensy (TeensyMaxMixer) to control:

Buttons:

- Stop
- Play/Pause
- A/B
- Forward
- Rewind
- Limiter On/Off
- Stereo Imager On/Off
- Saturation On/Off

Dials:

- Compressor Dials
- Saturation Dials
- Limiter Dials
- Stereo Imager Dials
- Gain
- Volume Slider

All of this I have decided to call 'TeensyMaxMixer', which is then sent to 'TeensyMaxMixerMain' which is what will be connected to the laptop/computer.

Using RX/TX of 0 and 1 respectively outputting all serial communication to input RX/TX of the Main Teensy, 31 and 32 respectively.

TeensyMaxMixerMain controls:

Buttons:

- Equaliser On/Off
- Gain
- EQ1
- C1
- SA
- EQ2
- C2
- SI
- L1
- L2
- Compressor On/Off

Dials:

- Equaliser Dials

Furthermore, I have connected the power supply to everything that the main mixer is controlling due to voltage drops, as discussed earlier to ensure we do not have a reoccurring problem again.

We can see all of this in the new and Final Wiring Diagram I have made in Figure 9.1.

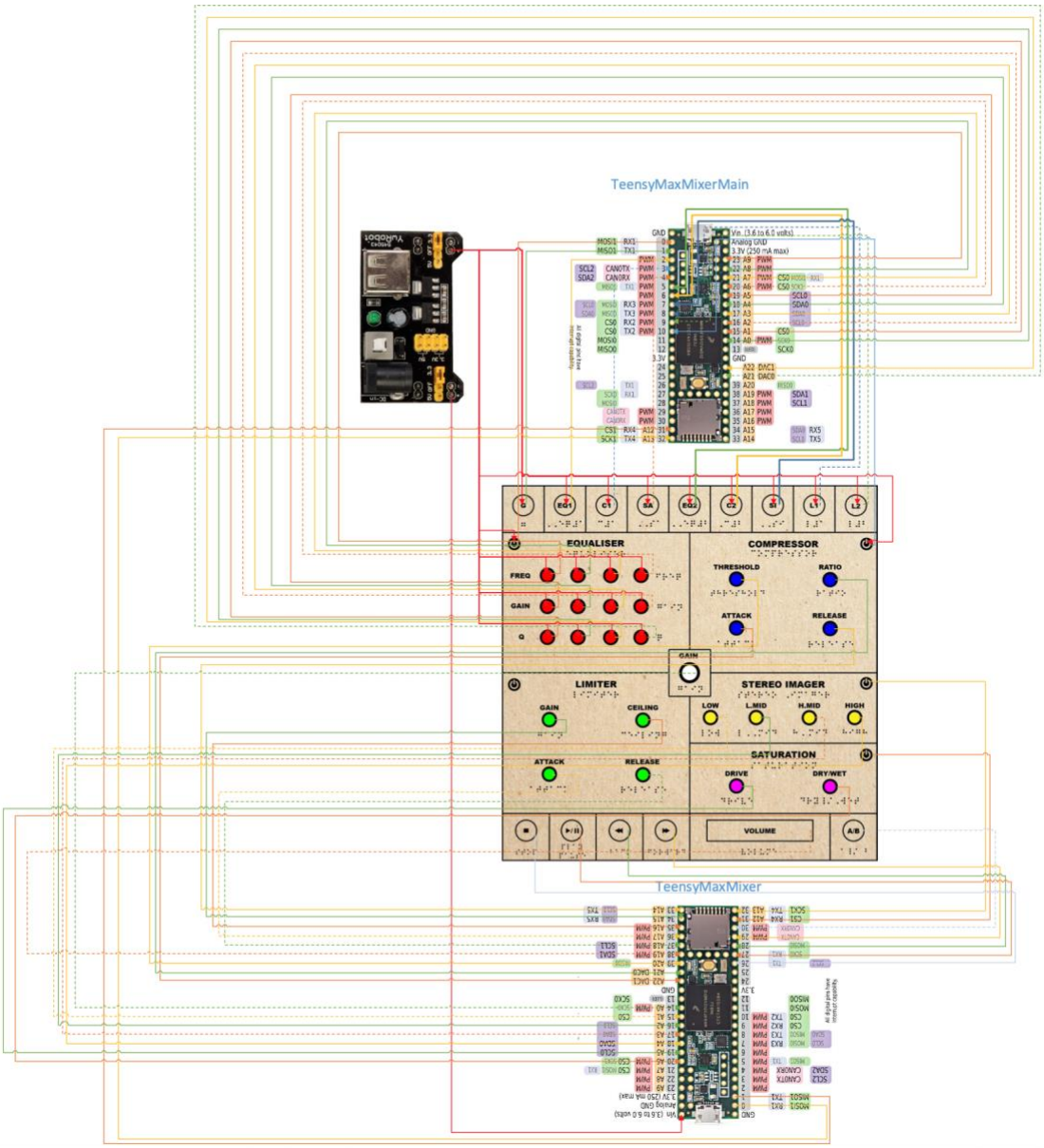

Figure 9.1 – Final Wiring Diagram

Code:

The first thing I did before I started is download the 'ArduinoJson', library created by Beniot Blanchon. I was informed that this will be the most feasible method of sending data between the two Teensys', using serilization and deserilization within the JSON library. I studied how to use this using multiple videos posted by Beniot Blanchon himself via YouTube, an e-Book created by Blanchon named 'Mastering ArduinoJson 6: Efficient JSON serialization embedded for C++, a few alternate channels and the ArduinoJson example files.

Links:

- https://www.youtube.com/watch?v=z_rPvQhKTfY
- https://www.youtube.com/watch?v=hAB4TdX8dwM&t=3518s
- https://www.youtube.com/watch?v=nfr6wddRRxo

The two main codes are:

- TeensyMaxMixer.ino
- TeensyMaxMixerMain.ino

The code starts with a class 'TeensyMixer', that will contain and store all int values (integer) in one place.

Then I needed to make a button object (another class), which is created firstly setting the void.begin – this is setting the mode as an input. Then a bool function named 'isButtonPressed', which outputs either true or false (hence why you receive a 0 or 1 within serial) and this is done by a digitalRead – if it is high then it will return a true, else false.

Instantiating TeensyMixer, all Button objects with their corresponding pins on Teensy and mixerJson. The mixerJson is another data structure, which packs all the values on TeensyMaxMixer.ino and gets sent to the other Teensy. This is named 'DynamicJsonDocument mixerJson (1024)'.

Then we move onto the void.setup, this begins with LED, to indicate the Teensy is on. Then we have a serial.begin for debugging and serial1.begin - this is responsible for sending it to the other Teensy and we have the '.begins' for every button which does the same thing as before (mode as input).

Next, we have void.loop, these are for the analogReads for every potentiometer, corresponding to the Analog inputs they have been connected to, in comparison to the buttons which are digitalRead. Here we also start making use of the class teensyMixer to access every pot/button variable, thus reading it and populating the object. Then straight after the mixerJson gets populated by all teensyMixer objects, Json is a key pair object, so you have the

key and then the value inside that, as shown in Figure 9.2. Lastly, ending with a serializeJson, which sends it across to serial1 (to the other teensy – TeensyMaxMixerMain). Furthermore, I have a debug for the serial monitor as extra.

```
mixerJson["compressorRatio"] = teensyMixer.compressorRatio;
mixerJson["compressorAttack"] = teensyMixer.compressorAttack;
mixerJson["comppressorRelease"] = teensyMixer.comppressorRelease;

mixerJson["limiterOnButton"] = teensyMixer.limiterOnButton;
mixerJson["limiterGain"] = teensyMixer.limiterGain;
mixerJson["limiterCieling"] = teensyMixer.limiterCieling;
mixerJson["limiterAttack"] = teensyMixer.limiterAttack;
mixerJson["limiterRelease"] = teensyMixer.limiterRelease;

mixerJson["stereoImagerOnButton"] = teensyMixer.stereoImagerOnButton;
mixerJson["stereoImagerLow"] = teensyMixer.stereoImagerLow;
mixerJson["stereoImagerLowMid"] = teensyMixer.stereoImagerLowMid;
mixerJson["stereoImagerHighMid"] = teensyMixer.stereoImagerHighMid;
mixerJson["stereoImagerHigh"] = teensyMixer.stereoImagerHigh;

mixerJson["saturationOnButton"] = teensyMixer.saturationOnButton;
mixerJson["saturationDrive"] = teensyMixer.saturationDrive;
mixerJson["saturationDryWet"] = teensyMixer.saturationDryWet;

mixerJson["gain"] = teensyMixer.gain;
mixerJson["volumeSlider"] = teensyMixer.volumeSlider;

mixerJson["ABButton"] = teensyMixer.ABButton;
mixerJson["fwdButton"] = teensyMixer.fwdButton;
mixerJson["prevButton"] = teensyMixer.prevButton;
mixerJson["playPauseButton"] = teensyMixer.playPauseButton;
mixerJson["stopButton"] = teensyMixer.stopButton;
mixerJson["limiterOnButton"] = teensyMixer.limiterOnButton;

serializeJson(mixerJson, Serial1);
//serializeJson(mixerJson, Serial);
//Serial.println("");
//debug();
```

Figure 9.2 – JSON keys with teensyMixer Objects & serialization

In the TeensyMaxMixerMain.ino (main Teensy which connects to device/software), this starts off with mixer.h which contains everything (all buttons and dials), then shows all the other remaining buttons which is all top buttons and EQ, Compressor on and off buttons which is instantiated using the pins. Next, we then have 'DynamicJsonDocument mixerJson (1024)' again, which now will be used as a container that we use to receive from other Teensy.

Our void.setup for this .ino is the same, with LED to indicate on and all buttons begins. Only difference is we have our standard serial.begin for debugging but now we also include serial4.begin, this is the receiving serial from the other teensy, it receives the Json on this serial.

Moving onto to the void loop, here we start with our DeserializationError err; this section receives from the TeensyMaxMixer with an if statement, where if the serial is available then it gets dumped into the Json object.

Then I set up the isButtonPressed and analogReads all in order of:

1. Top Buttons
2. Equaliser
3. Compressor (On and Off button only)

4. ***From here I use another if statement to allow me to extract the mixerJson using the corresponding key from the previous .ino, which then assigns the value to the corresponding mixer object.***

Lastly, I made a sendMixerToMax object, this contains a void Mixer. Instantiating a string object, building each section one by one (for every dial/pot) and then serial.println at the end, all shown in Figure 9.3.

```
if (err == DeserializationError::Ok)
{
    mixer.thresholdAnalogue     = mixerJson["compressorThreshold"].as<int>();
    mixer.ratioAnalogue         = mixerJson["compressorRatio"].as<int>();
    mixer.attackAnalogue        = mixerJson["compressorAttack"].as<int>();
    mixer.releaseAnalogue       = mixerJson["comppressorRelease"].as<int>();

    mixer.limiterOnButton       = mixerJson["limiterOnButton"].as<int>();
    mixer.limiterGainAnalogue   = mixerJson["limiterGain"].as<int>();
    mixer.limiterCielingAnalogue= mixerJson["limiterCieling"].as<int>();
    mixer.limiterAttackAnalogue = mixerJson["limiterAttack"].as<int>();
    mixer.limiterReleaseAnalogue= mixerJson["limiterRelease"].as<int>();

    mixer.stereoImagerOnButton  = mixerJson["stereoImagerOnButton"].as<int>();
    mixer.stereoLowAnalogue     = mixerJson["stereoImagerLow"].as<int>();
    mixer.stereoLmidAnalogue    = mixerJson["stereoImagerLowMid"].as<int>();
    mixer.stereoHmidAnalogue    = mixerJson["stereoImagerHighMid"].as<int>();
    mixer.stereoHighAnalogue    = mixerJson["stereoImagerHigh"].as<int>();

    mixer.saturationOnButton    = mixerJson["saturationOnButton"].as<int>();
    mixer.saturationDrive       = mixerJson["saturationDrive"].as<int>();
    mixer.saturationDryWet      = mixerJson["saturationDryWet"].as<int>();

    mixer.Gain                  = mixerJson["gain"].as<int>();
    mixer.volumeSilder          = mixerJson["volumeSlider"].as<int>();


    mixer.ABButton              = mixerJson["ABButton"].as<int>();
    mixer.fwdButton             = mixerJson["fwdButton"].as<int>();
    mixer.prevButton            = mixerJson["prevButton"].as<int>();
    mixer.playPauseButton       = mixerJson["playPauseButton"].as<int>();
    mixer.stopButton            = mixerJson["stopButton"].as<int>();
    mixer.limiterOnButton       = mixerJson["limiterOnButton"].as<int>();

}

mixer.sendMixerToMax();
```

```
class Mixer
{
public:
    Mixer();

    ~Mixer();

    void sendMixerToMax();

    String buildTopButtons();
    String buildEqualiser();
    String buildCompressor();
    String buildLimiter();
    String buildStereoImager();
    String buildSaturation();
    String buildButtomButtonsAndVolSlider();

    // top buttons
    int GButton;
    int EQ1Button;
    int C1Button;
    int SAButton;
    int EQ2Button;
    int C2Button;
    int SIButton;
    int L1Button;
    int L2Button;

    // equaliser
    int equaliserOnButton;
    int freqAnalogueLeft;
    int freqAnalogueMidLeft;
    int freqAnalogueMidRight;
    int freqAnalogueRight;
```

Figure 9.3 – sendMixerToMax object

Each section has its own build of string (e.g. as shown in Figure 9.3 – "buildTopButtons"), this can be found in mixer.cpp, where we have the main string objects code. In Figure 9.4 we can see it gets the values and converts it

into a string and then you have a space in-between each value, using " ", this is for all sections. Finally, sending it back to serial port for printing.

```
String Mixer::buildTopButtons()
{
    return " " + String(GButton)
    + " " + String(EQ1Button)
    + " " + String(C1Button)
    + " " + String(SAButton)
    + " " + String(EQ2Button)
    + " " + String(C2Button)
    + " " + String(SIButton)
    + " " + String(L1Button)
    + " " + String(L2Button);
}

String Mixer::buildEqualiser()
{
    return " " + String(equaliserOnButton)
    + " " + String(freqAnalogueLeft)
    + " " + String(freqAnalogueMidLeft)
    + " " + String(freqAnalogueMidRight)
    + " " + String(freqAnalogueRight)

    + " " + String(gainAnalogueLeft)
    + " " + String(gainAnalogueMidLeft)
    + " " + String(gainAnalogueMidRight)
    + " " + String(gainAnalogueRight)

    + " " + String(QAnalogueLeft)
    + " " + String(QAnalogueMidLeft)
    + " " + String(QAnalogueMidRight)
    + " " + String(QAnalogueRight);
}

String Mixer::buildCompressor()
{
    return " " + String(compressonOnButton)
    + " " + String(thresholdAnalogue)
    + " " + String(ratioAnalogue)
    + " " + String(attackAnalogue)
```

Figure 9.4 – String values with spaces

Previous Code (Failed):

This code can be found in folder '***Previous Code (Inconsistent)***' in final hand in.

The through-hole multiplexer (74HC4067 16-input multiplexer) works by connecting VCC to 5V, GND and E (Enable Pin) to Ground, 4 control pins (S0, S1, S2, S3) to access all 16 inputs (which connect to digital PWM~ pins on Arduino) and lastly connecting COMMON INPUT/OUTPUT pin (pin 1) sometimes classed as 'Z' pin to the Analog pin – as we are spreading the 28 potentiometers across the two multiplexers evenly. This also makes the multiplexers analog only, as it can't be both digital and analog, since we have more analog inputs to spread, I chose to use the multiplexers for this. In terms of the digital touch sensors there is enough inputs on the Arduino Mega for this.

This same process was repeated for 'MUX 2' – everything stated above can be shown in my original proposed wiring diagram, Figure 1.4. Furthermore, 'Enable' pins are connected to ground on the breadboard, as it is missing from diagram.

In the diagram we can see I spread the controls by 14 each:

Mux 1:

- EQ Pots
- Saturation Pots

Mux 2:

- Compressor Pots
- Limiter Pots
- Stereo Imager Pots
- Gain Pot
- Slider Pot (Volume)

This can be seen in the code MaxMixer.ino.

Creating a multiplexer class 'Multiplexer.h' along with Multiplexer.cpp library that was found online, which corresponds to the function table (truth table) provided in the final hand in as "***74HC4067 NXP Semiconductors Datasheet***". When testing I found that some of the channels were not working on **both** integrated multiplexers. As I ordered CD74HC4067 MUX Breakout boards at the same time, as spares. I tested these as well and found that all channels worked and used code found via sparkfun.com.

This all connects to the MaxMixer.ino, which has a similar layout as my final TeensyMaxMixerMain code. Firstly, creating defined constants for each potentiometer that will be connected and both MUX control pins, as shown in Figure 9.5.

```
#define mux1CtrlPin1 8
#define mux1ctrlPin2 9
#define mux1ctrlPin3 10
#define mux1ctrlPin4 11
#define mux1analogReadPin A0

#define mux2CtrlPin1 2
#define mux2ctrlPin2 3
#define mux2ctrlPin3 4
#define mux2ctrlPin4 5
#define mux2analogReadPin A1
```

Figure 9.5 – Defining constants

Then there is the Mux multiplexer1 and Mux multiplexer2, which are both containers for the ctrlspins.

Next, making button objects, then when looking at the void setup, which has a serial.begin (9600) and '.begins' for every button which sets the mode as input, as all previously mentioned in final code as well.

Lastly, moving onto the void loop which again is similar, as we have our mixer.h which contains all buttons and pots, in the void loop we have our analog reads from each multiplexer channel, which the value gets assigned to the mixer.xxx (where 'xxx' will be a potentiometer), as shown in Figure 9.6.

```
// Equaliser
mixer.equaliserOnButton     = EqualizserOnButton.isButtonPressed();

mixer.freqAnalogueLeft      = multiplexer1.read(chanFreqAnalogueLeft);
mixer.freqAnalogueMidLeft   = multiplexer1.read(chanFreqAnalogueMidLeft);
mixer.freqAnalogueMidRight  = multiplexer1.read(chanFreqAnalogueMidRight);
mixer.freqAnalogueRight     = multiplexer1.read(chanFreqAnalogueRight);

mixer.gainAnalogueLeft      = multiplexer1.read(chanGainAnalogueLeft);
mixer.gainAnalogueMidLeft   = multiplexer1.read(chanGainAnalogueMidLeft);
mixer.gainAnalogueMidRight  = multiplexer1.read(chanGainAnalogueMidRight);
mixer.gainAnalogueRight     = multiplexer1.read(chanGainAnalogueRight);

mixer.QAnalogueLeft         = multiplexer1.read(chanQAnalogueLeft);
mixer.QAnalogueMidLeft      = multiplexer1.read(chanQAnalogueMidLeft);
mixer.QAnalogueMidRight     = multiplexer1.read(chanQAnalogueMidRight);
mixer.QAnalogueRight        = multiplexer1.read(chanQAnalogueRight);
```

Figure 9.6 – Populating mixer

This then ends in a mixer.sendMixerToMax, which works exactly like Figure 9.3 creating strings and then followed on by Figure 9.4, which then gets sent to the serial port for printing.

User Manual

1. Connect the device to a PC or Laptop and mains. (Using provided Micro USB cable & power supply)
2. Open 'KMC' Application.
3. For LOAD MASTER section press, **OPEN** first, then **REPLACE**, then the **BUTTON** on the right-hand side next to replace. Repeat process for LOAD REFERENCE.

4. Then turn on Volume button with the speaker sign highlighted in off white.

5. Now you can press **STOP** or **PLAY**, and your track should start playing.
6. **A/B** is the button to change from your Master to Reference track.
7. To export your master once you have finished, please ensure you have selected your master (**A**) and not reference track (**B**), then press **SAVE** to name your file, then press **RECORD** and the red timer should start moving, as an indicator.

Important Guidelines:

- Whilst exporting your master, everything works in real-time, **DO NOT** make changes to any parameter whilst this is happening unless you specifically want automation to happen on a particular parameter on your final master.
- **Only if** you are using the hardware controller with the application, please press  **before** loading your master and reference, this allows time for the application to sync with the hardware parameters.
- Press **open** button if you want to change audio drivers.

Application

In this section I will be doing the user testing with the plugin, I will set out a task for each user to master the same track using K Mastering Console solely.

Comparing the results using analyzers, metering, and references, I will also ask for a review on the product itself giving positives and negatives (what can be improved). In addition, noting down the plugins that they used in their final master.

User 1 and **User 2** will be compared to each other, and the already professionally Mastered track done elsewhere (not using the plugin). Furthermore, both Users used the same reference track which was 'Sheff G – Weight On Me (feat. Sleepy Hallow)'.

Track Name: Sanji
Artist: TJ
Producer: KESH (Keshav Deb)
Mix & Master: Turkish Dcypha

User 1 (Me):

Chain used in plugin:

- NLS Buss (Always On)
- FF Pro-Q3
- SSL Comp
- FF Saturn
- AR TG Mastering
- MStereoProcessor
- L2
- FF Pro-L2

For this I went for the serial limiting technique method with EQ'ing first then some further EQ after compression and saturation.

Integrated LUFS: -11.3

User 2 (Audio Engineer – Duramaney Kamara)

Chain used in plugin:

- NLS Buss (Always On)
- FF Pro-Q3
- SSL Comp
- FF Saturn
- VComp
- AR TG Mastering
- MStereoProcessor
- FF Pro-L2

For this they used all processors excluding the second limiter, I believe this was done due to the extra use of both compressors, so the extra limiting was not needed, as the Pro-L2 was enough to handle the role of limiting.

Integrated LUFS: -11.3

Review: "I found this to be quite a challenging and interesting task, as not realizing how much visual reference I would usually use and quickly apply my own presets to then customize to the specific track. It really made me hone into using my ears to get the job done. One thing I would change is the VComp and could maybe even allow the user to choose which plugins that will be used in that specific signal chain. It would be nice to also have one M-S EQ and one stereo. Lastly introducing multiband compression will be very useful in certain cases."

Professional Master (Turkish Dcypha)

Integrated LUFS: -11.2

We can see in terms of LUFS that there they are almost identical. However, when listening to them and even in Figure 10.0, we can see and hear that the professional master has a slightly louder perceived loudness.

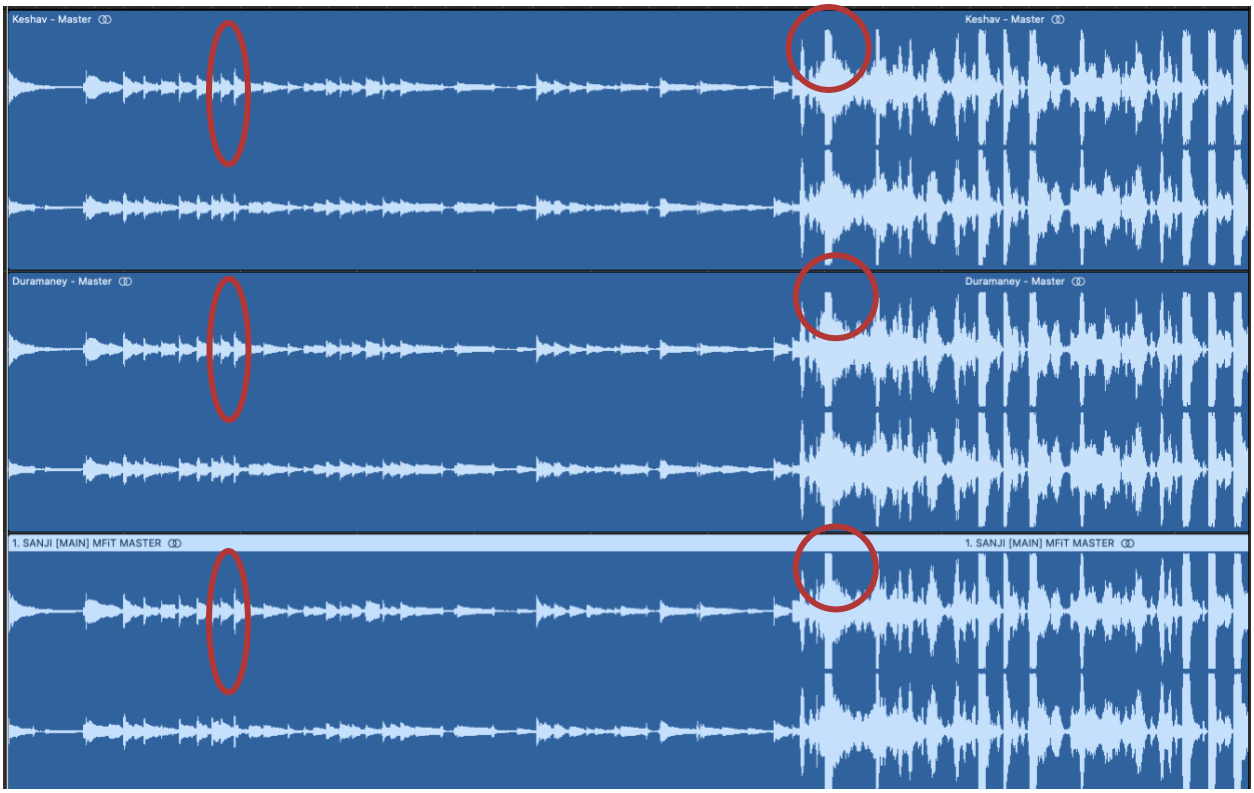

Figure 10.0 – Master Comparisons

Here visually we can see, especially in the intro section there the waveform is slightly thicker (louder), even with certain spikes, as shown in Figure 10.0 and how when the kick comes in on the master it is doing full brick-wall limiting. Whereas User 2 is slightly less of a full brick-wall and User 1 is even less compared to User 2, as we can see it rounding off (where circled).

When listening to the three, the professional master does have that slight edge in loudness. You can find this logic project with all three songs in folder "**Master Comparisons**".

Listening to the busiest section bar 40 to 57 (differences between each other):

- User 1 vocals sound more muddier compared to the other two.
- User 2 vocals sounds much more upfront, and present compared to other two (with more low-mid presence).
- Professional master does sound upfront but compared to User 2 there is slightly less body and clarity in vocals.

- Professional master has more consistency in terms of the level of the vocals, this may be due to the compression settings effecting certain frequencies in User 1 and 2 masters.
- Bass response in User 2 personally sounds much cleaner in comparison to professional and User 1 (this may also be another reason as to why the vocals sound like they are being squashed/quieter).

Note: All these comparisons are subject to my listening experience of all three on several monitors. Please use folder/logic project "**Masters Comparison**" and also folder "**Master Audio Assets**" to make your judgement between the tracks.

Contextualization

This idea was to create a new approach within mastering, focusing on the aspects of active listening, without having any visual influence. As with most plugins/hardware units there is always numerical values to go with a parameter or a visual analyzer being displayed. But with K Mastering Console, it allows you to hone into really using your ears to hear the difference that is being made without knowing how much you have pushed something.

From tutors to my own learning and research, I kept reading and getting told the importance of using your ears and even training them to be able to identify different frequencies, common problematic areas and much more. It is the most essential thing when it comes to mixing and mastering, "To get a truly great mix you have to hear every single detail of the track you are working on" as quoted by (Hahn, 2020), this also translates in mastering terms. There are also many other factors which aid to improve your listening skills, this ranges from room acoustics/treatment to equipment placement (as explained by (Genelec Monitor Setup Guide, 2018)). You can be tricked by relying too much on analyzers, when making important EQ decisions (Hahn, 2020), hence why people do tend to turn off the visual display or look away when adjusting the sound.

This all reflects my idea of creating a product that will add to the list of things that will help the consumer train their ears. I believe there is a gap in the market for a product like this, that not only focuses on having an all-in-one mastering unit but educating new and experienced mastering engineers via the lack of relying on visual feedback. Furthermore, I have taken this a step further by creating a hardware controller that will replicate the software and through this I have decided to implement braille. This leads into a whole new consumer market for blind/visually impaired people to have a new hobby or career within the music industry, something of which I feel there is a lack of and could be a big demand. "Studies show that blind persons perform nonvisual task better than those with sight…including hearing…five of the blind participants could accurately localize sounds monaurally; most of the sighted could not", as stated in (Loss of Sight and Enhanced Hearing: A Neural Picture, 2005) – this shows that it will be something the blind can perform very well at to an extraordinary standard just like we have seen in renowned musicians like Stevie Wonder and Ray Charles. I believe the main possible reason behind the enhanced hearing is

- the fact that they are having to rely on that sense the most, to locate, in conjunction with their touch sense.

Lastly, it can be implemented into special needs schools across the world as part of their curriculum, providing another alternative with a product like this.

Conclusion

In conclusion, I believe the project overall was a success in terms of everything working with the plugin and the prototype hardware. However, there would be a few things that I would change if I was to do this again.

1. Research into other microcontrollers from the beginning
2. Make a more revised plan
3. Design hardware prototype with more thought and get a custom-built casing for design to be more alike with illustration
4. Time management of plugin and hardware build
5. Add the option of being able to choose from a list of plugins
6. Adding multi-band compression or other processors
7. Adding presets
8. Building a dedicated PCB for the entire build
9. Get more people to test the plugin
10. Get people to test the hardware

With all the improvements that could be made, it could then lead into less faults and trail-and-error, leading to a more versatile and cleaner product. I believe that there is a gap in the market for this concept with also the inclusion of more blind/visually impaired friendly equipment in the industry. Furthermore, with SSL pushing the dedicated controllers for mixing, they have also released a new product this year, which is the UC1 (a dedicated controller for their Chanel Strip 2 and Bus Compressor 2. There has not been a dedicated controller for mastering yet.

Lastly, this product could push the idea of more 'in-the-box' mastering but using an actual physical product and not just mouse and keyboard. This could also be translated into and ear training machine/course that could be embedded into the software, maybe before your mastering sessions to 'warm' your ears up,

there could be potentially a 5-min exercise to test you, as I am also pushing the idea of using our ears.

Links

- K Mastering Console & Hardware – Overview: https://youtu.be/7VcPg2mcf24
- Proof testing all dials & buttons: https://youtu.be/lVlQXRFgmVo
- My website: www.keshxv.com
- Blog: https://keshavdeb.wixsite.com/fmpblog

Bibliography

Afb.org. n.d. *What Is Braille?* | *American Foundation for the Blind*. [online] Available at: <https://www.afb.org/blindness-and-low-vision/braille/what-braille>.

Audio Issues. n.d. *What is a Limiter and How to Use One to Make Better Masters From Your Home Studio - Audio Issues*. [online] Available at: <https://www.audio-issues.com/music-mixing/what-is-a-limiter-how-to-make-better-mixes-mastering/>.

Downloads.ctfassets.net. 2018. *Genelec Monitor Setup Guide*. [online] Available at: <https://downloads.ctfassets.net/4zjnzn055a4v/59uPaa9QITsA6tsqhlOv5T/8254aa9ca070ecd1090ecd16b202396b/Monitor_setup_guide_BBAGE125e-lightweight.pdf>.

Hahn, M., 2020. *Hard Truths: You're Listening to Your Mix Wrong*. [online] LANDR Blog. Available at: <https://blog.landr.com/listening-habits/>.

Inglis, S., 2021. *SSL UF8*. [online] Soundonsound.com. Available at: <https://www.soundonsound.com/reviews/ssl-uf8>.

iZotope. (2014). *What Is Mastering and Why Is It Important?*. [online] Available at: <https://www.izotope.com/en/learn/what-is-mastering.html>.

Kagan, A., 2020. *Pro Mastering Tips: Compression Pt. I*. [online] sonarworks. Available at: <https://www.sonarworks.com/soundid-reference/blog/learn/pro-mastering-tips-compression-pt-i/>.

Katz, R., 2002. *Mastering Audio: the art and the science*. Boston: Focal Press, p.111.

Katz, R., 2002. *Mastering Audio: the art and the science*. Boston: Focal Press, p.121.

Keeley, E., 2017. *What is Linear Phase EQ? And Why It's Great for Mastering* | *MasteringBOX*. [online] MasteringBOX. Available at: <https://www.masteringbox.com/linear-phase-eq-great-mastering/#:~:text=Linear%20phase%20EQ%20can%20be,when%20mastering%2C%20not%20when%20mixing.>.

Mastering The Mix. 2017. *How To EQ During Mastering*. [online] Available at: <https://www.masteringthemix.com/blogs/learn/how-to-eq-during-mastering>.

Messitte, N., 2018. *An Introduction to Limiters (and How to Use Them)*. [online] iZotope. Available at: <https://www.izotope.com/en/learn/an-introduction-to-limiters-and-how-to-use-them.html>.

Owsinski, B., (2017). *Mastering engineer's handbook 4th edition*. 4th ed. BOBBY OWSINSKI MEDIA GROUP, p.17-18

Pack, B., 2020. *What Is Serial Processing And How To Use It*. [online] Available at: <https://www.sonarworks.com/soundid-reference/blog/learn/what-is-serial-processing-and-how-to-use-it/>.

Perkins School for the Blind. n.d. *12 things you probably don't know about braille*. [online] Available at: <https://www.perkins.org/stories/12-things-you-probably-dont-know-about-braille#:~:text=There%20are%20two%20versions%20of%20braille%20%E2%80%93%20contracted%20and%20uncontracted.>.

PLoS Biology, 2005. Loss of Sight and Enhanced Hearing: A Neural Picture. [online] 3(2), p.e48. Available at: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC544930/>.

Raman, R., 2019. *Mastering 101: Compression - Blog | Splice*. [online] Splice.com. Available at: <https://splice.com/blog/mastering-101-compression/>.

RNIB - See differently. n.d. *Contracted (grade 2) braille explained*. [online] Available at: <https://www.rnib.org.uk/braille-and-moon-tactile-codes/braille-codes/contracted-grade-2-braille-explained#:~:text=Contracted%20(grade%202)%20braille%20is,like%20a%20kind%20of%20shorthand.>.

RNIB - See differently. n.d. *Unified English Braille (UEB)*. [online] Available at: <https://www.rnib.org.uk/braille-and-moon-%E2%80%93-tactile-codes-braille-codes/unified-english-braille-ueb>.

Sageaudio.com. n.d. *Best Mastering Chain — Sage Audio*. [online] Available at: <https://www.sageaudio.com/blog/mastering/best-mastering-chain.php>.

Sageaudio.com. n.d. *How to Use Stereo Imaging During Mastering*. [online] Available at: <https://www.sageaudio.com/blog/mastering/how-to-use-stereo-imaging-during-mixing-before-mastering.php>.

Sageaudio.com. n.d. *What is Saturation for Mixing and Mastering? — Sage Audio*. [online] Available at: <https://www.sageaudio.com/blog/mastering/what-is-saturation-for-mixing-and-mastering.php#:~:text=Saturation%20is%20used%20during%20mastering,for%20creating%20a%20competitive%20sound.>.

Simpson, C., 2013. *The rules of unified English Braille*. 2nd ed. International Council of English Braille, p.xi.

Soundonsound.com. 2016. *IK Multimedia releases Lurssen Mastering Console*. [online] Available at: <https://www.soundonsound.com/news/ik-multimedia-releases-lurssen-mastering-console>.

Soundways.com. n.d. *Mastering: How to Set a Limiter | Soundways*. [online] Available at: <http://www.soundways.com/content/mastering-how-set-limiter>.

Swisher, D., n.d. *Linear Phase EQ: The Dos and Don'ts of Linear EQ*. [online] Musician on a Mission. Available at: <https://www.musicianonamission.com/linear-phase-eq/>.

waves.com. 2017. *10 Tips for Effective EQ during Mastering | Waves*. [online] Available at: <https://www.waves.com/10-tips-effective-eq-mastering>.

Zambrano, O., 2019. *What Is an Ideal Mastering Signal Chain?*. [online] iZotope. Available at: <https://www.izotope.com/en/learn/what-is-an-ideal-mastering-signal-chain.html>.

Figures